DOCUMENTATION DU CODE

MoMoGuard — explication fichier par fichier

Date : 26/06/2026
Ce document décrit chaque fichier source, chaque fonction déclarée et chaque ligne de code avec son rôle.

Table des matières

• run.py — Point d'entrée — lance le serveur central.
• server/app.py — Serveur Flask : routes, auth admin, API agents, WebSocket.
• server/database.py — Couche SQLAlchemy : tables, requêtes, conseils.
• server/scoring.py — Moteur de détection par règles explicables.
• server/threat_intel.py — Vérification URL via Google Safe Browsing.
• server/firebase_auth.py — Auth mobile Firebase + notifications push.
• agents/windows_agent.py — Agent Windows : presse-papier, navigateur, fichiers.
• server/static/dashboard.js — Dashboard temps réel (JavaScript).
• scripts/seed_demo.py — Insertion de données de démonstration.
• scripts/backup_db.py — Sauvegarde JSON de la base.
• evaluation/evaluer.py — Évaluation chiffrée du moteur de scoring.

Fichier : run.py

Point d'entrée — lance le serveur central.

Explication ligne par ligne

L.	Code	Explication
1	`"""Point d'entrée MoMoGuard — lance le serveur central."""`	Commentaire / docstring : """Point d'entrée MoMoGuard — lance le serveur central."""
2	`from server.app import main`	Import de module ou symbole Python.
3		Ligne vide (séparation visuelle).
4	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
5	`    main()`	Instruction Python.

Fichier : server/app.py

Serveur Flask : routes, auth admin, API agents, WebSocket.

Fonctions déclarées

_entetes_securite(response) — ligne 47 [app.after_request]

→ Ajoute des en-têtes de sécurité à toutes les réponses.

ensure_default_admin() — ligne 65

→ Crée un administrateur par défaut au premier démarrage.

login_required(view) — ligne 75

→ Fonction utilitaire « login_required ».

_audit(action, target, details) — ligne 84

→ Trace une action administrateur dans le journal d'audit.

login() — ligne 93 [app.route('/login', methods=['GET', 'POST']), limiter.limit('10 per minute', methods=['POST'])]

→ Page et traitement de connexion administrateur.

login_2fa() — ligne 111 [app.route('/login/2fa', methods=['GET', 'POST']), limiter.limit('10 per minute', methods=['POST'])]

→ Vérification du code TOTP (2FA).

logout() — ligne 129 [app.route('/logout')]

→ Déconnexion et destruction de session.

account() — ligne 137 [app.route('/account', methods=['GET', 'POST']), login_required]

→ Gestion du compte admin (mot de passe).

_qr_svg(data) — ligne 158

→ Génère un QR code SVG (inline, sans dépendance image).

twofa_setup() — ligne 171 [app.route('/account/2fa/setup'), login_required]

→ Fonction utilitaire « twofa_setup ».

twofa_enable() — ligne 183 [app.route('/account/2fa/enable', methods=['POST']), login_required]

→ Fonction utilitaire « twofa_enable ».

twofa_disable() — ligne 201 [app.route('/account/2fa/disable', methods=['POST']), login_required]

→ Fonction utilitaire « twofa_disable ».

dashboard() — ligne 211 [app.route('/'), login_required]

→ Page principale de supervision des alertes.

demo_alert() — ligne 229 [app.route('/demo', methods=['POST']), login_required]

→ Génère une alerte frauduleuse de démonstration (utile en soutenance).

api_reports() — ligne 246 [app.route('/api/reports'), login_required]

→ Fonction utilitaire « api_reports ».

report() — ligne 258 [app.route('/report', methods=['POST']), csrf.exempt, limiter.limit('120 per minute')]

→ Reçoit et traite un rapport d'agent (POST /report).

_annotate_online(agents, window_seconds) — ligne 332

→ Marque chaque agent en ligne s'il a été vu dans la fenêtre donnée.

agents() — ligne 351 [app.route('/agents', methods=['GET', 'POST']), login_required]

→ Fonction utilitaire « agents ».

agent_block(agent_id) — ligne 370 [app.route('/agents/<agent_id>/block', methods=['POST']), login_required]

→ Fonction utilitaire « agent_block ».

agent_unblock(agent_id) — ligne 378 [app.route('/agents/<agent_id>/unblock', methods=['POST']), login_required]

→ Fonction utilitaire « agent_unblock ».

agent_remove(agent_id) — ligne 386 [app.route('/agents/<agent_id>/remove', methods=['POST']), login_required]

→ Fonction utilitaire « agent_remove ».

auth_firebase() — ligne 396 [app.route('/api/auth/firebase', methods=['POST']), csrf.exempt]

→ L'app envoie son ID token Firebase. On le vérifie, puis on crée/relie
l'utilisateur dans MySQL (statut 'pending' tant que l'admin n'a pas approuvé).

my_reports() — ligne 430 [app.route('/api/my-reports', methods=['POST']), csrf.exempt]

→ L'app envoie son ID token ; on renvoie les alertes de CE client.

fcm_token() — ligne 458 [app.route('/api/fcm-token', methods=['POST']), csrf.exempt]

→ L'app enregistre (ou met à jour) son jeton FCM pour recevoir les push.

users() — ligne 479 [app.route('/users'), login_required]

→ Fonction utilitaire « users ».

user_approve(uid) — ligne 485 [app.route('/users/<uid>/approve', methods=['POST']), login_required]

→ Fonction utilitaire « user_approve ».

user_reject(uid) — ligne 493 [app.route('/users/<uid>/reject', methods=['POST']), login_required]

→ Fonction utilitaire « user_reject ».

user_block(uid) — ligne 501 [app.route('/users/<uid>/block', methods=['POST']), login_required]

→ Fonction utilitaire « user_block ».

user_detail(uid) — ligne 509 [app.route('/users/<uid>'), login_required]

→ Fonction utilitaire « user_detail ».

user_edit(uid) — ligne 518 [app.route('/users/<uid>/edit', methods=['POST']), login_required]

→ Fonction utilitaire « user_edit ».

user_password(uid) — ligne 527 [app.route('/users/<uid>/password', methods=['POST']), login_required]

→ Fonction utilitaire « user_password ».

settings_page() — ligne 563 [app.route('/settings', methods=['GET', 'POST']), login_required]

→ Fonction utilitaire « settings_page ».

consent() — ligne 595 [app.route('/consent', methods=['GET', 'POST'])]

→ Fonction utilitaire « consent ».

consents() — ligne 614 [app.route('/consents'), login_required]

→ Fonction utilitaire « consents ».

consent_revoke(cid) — ligne 620 [app.route('/consent/<int:cid>/revoke', methods=['POST']), login_required]

→ Fonction utilitaire « consent_revoke ».

audit() — ligne 628 [app.route('/audit'), login_required]

→ Fonction utilitaire « audit ».

on_connect() — ligne 633 [socketio.on('connect')]

→ Fonction utilitaire « on_connect ».

main() — ligne 637

→ Initialise la base, l'admin et démarre le serveur.

wrapped() — ligne 77 [wraps(view)]

→ Fonction utilitaire « wrapped ».

Explication ligne par ligne

L.	Code	Explication
1	`"""Serveur central MoMoGuard : Flask + Flask-SocketIO + SQLite.`	Commentaire / docstring : """Serveur central MoMoGuard : Flask + Flask-SocketIO + SQLite.
2		Ligne vide (séparation visuelle).
3	`Ajouts de cette version :`	Instruction Python.
4	`  - compte administrateur (connexion, déconnexion, changement de mot de passe) ;`	Instruction Python.
5	`  - gestion des agents (enregistrer, bloquer/débloquer, retirer) ;`	Instruction Python.
6	`  - les rapports d'un agent bloqué sont refusés, et un agent retiré peut s'enregistrer à nouveau ;`	Instruction Python.
7	`  - chaque rapport signé doit désormais être accompagné d'un jeton TOTP, et le serveur vérifie que`	Instruction Python.
8	`    un jeton doit le fournir.`	Instruction Python.
9	`"""`	Commentaire / docstring : """
10		Ligne vide (séparation visuelle).
11	`import os`	Import de module ou symbole Python.
12	`import secrets`	Import de module ou symbole Python.
13	`from datetime import datetime, timezone`	Import de module ou symbole Python.
14	`from functools import wraps`	Import de module ou symbole Python.
15		Ligne vide (séparation visuelle).
16	`from flask import (Flask, request, jsonify, render_template, redirect,`	Import de module ou symbole Python.
17	`                   url_for, session, flash)`	Instruction Python.
18	`from flask_socketio import SocketIO`	Import de module ou symbole Python.

L.	Code	Explication
19	`from werkzeug.security import generate_password_hash, check_password_hash`	Import de module ou symbole Python.
20	`from flask_wtf.csrf import CSRFProtect`	Import de module ou symbole Python.
21	`from flask_limiter import Limiter`	Import de module ou symbole Python.
22	`from flask_limiter.util import get_remote_address`	Import de module ou symbole Python.
23		Ligne vide (séparation visuelle).
24	`import pyotp`	Import de module ou symbole Python.

L.	Code	Explication
25	`from . import database, scoring, firebase_auth, threat_intel`	Import de module ou symbole Python.
26		Ligne vide (séparation visuelle).
27	`app = Flask(__name__)`	Affectation de variable ou constante.
28	`# Clé secrète stable si fournie (sinon générée — sessions/CSRF perdus au redémarrage)`	Commentaire / docstring : Clé secrète stable si fournie (sinon générée — sessions/CSRF perdus au red
29	`app.config["SECRET_KEY"] = os.environ.get("MOMOGUARD_SECRET_KEY", secrets.token_hex(16))`	Configuration de l'application Flask.
30	`# Durcissement des cookies de session`	Commentaire / docstring : Durcissement des cookies de session
31	`app.config.update(`	Configuration de l'application Flask.
32	`    SESSION_COOKIE_HTTPONLY=True,`	Affectation de variable ou constante.
33	`    SESSION_COOKIE_SAMESITE="Lax",`	Affectation de variable ou constante.
34	`)`	Instruction Python.
35		Ligne vide (séparation visuelle).
36	`socketio = SocketIO(app, cors_allowed_origins="*", async_mode="threading")`	Affectation de variable ou constante.

L.	Code	Explication
37		Ligne vide (séparation visuelle).
38	`# Protection CSRF des formulaires (les API JSON des agents sont exemptées plus bas)`	Commentaire / docstring : Protection CSRF des formulaires (les API JSON des agents sont exemptées p
39	`csrf = CSRFProtect(app)`	Affectation de variable ou constante.
40		Ligne vide (séparation visuelle).
41	`# Limitation de débit : anti-force brute sur la connexion, anti-abus sur /report`	Commentaire / docstring : Limitation de débit : anti-force brute sur la connexion, anti-abus sur /report
42	`limiter = Limiter(get_remote_address, app=app, storage_uri="memory://",`	Affectation de variable ou constante.
43	`                  default_limits=[])`	Affectation de variable ou constante.
44		Ligne vide (séparation visuelle).
45		Ligne vide (séparation visuelle).
46	`@app.after_request`	Hook Flask exécuté après chaque requête.
47	`def _entetes_securite(response):`	Déclaration de la fonction _entetes_securite().
48	`    """Ajoute des en-têtes de sécurité à toutes les réponses."""`	Commentaire / docstring : """Ajoute des en-têtes de sécurité à toutes les réponses."""
49	`    response.headers["X-Content-Type-Options"] = "nosniff"`	Affectation de variable ou constante.
50	`    response.headers["X-Frame-Options"] = "DENY"`	Affectation de variable ou constante.
51	`    response.headers["Referrer-Policy"] = "no-referrer"`	Affectation de variable ou constante.
52	`    response.headers["Content-Security-Policy"] = (`	Affectation de variable ou constante.
53	`        "default-src 'self'; "`	Instruction Python.
54	`        "script-src 'self' https://cdnjs.cloudflare.com 'unsafe-inline'; "`	Instruction Python.

L.	Code	Explication
55	`        "style-src 'self' 'unsafe-inline'; "`	Instruction Python.
56	`        "connect-src 'self' ws: wss:; "`	Instruction Python.
57	`        "img-src 'self' data:;"`	Instruction Python.
58	`    )`	Instruction Python.
59	`    return response`	Retourne une valeur au appelant.
60		Ligne vide (séparation visuelle).
61	`DEFAULT_ADMIN_USER = "admin"`	Affectation de variable ou constante.
62	`DEFAULT_ADMIN_PASS = "momoguard"`	Affectation de variable ou constante.
63		Ligne vide (séparation visuelle).
64		Ligne vide (séparation visuelle).
65	`def ensure_default_admin():`	Déclaration de la fonction ensure_default_admin().
66	`    """Crée un administrateur par défaut au premier démarrage."""`	Commentaire / docstring : """Crée un administrateur par défaut au premier démarrage."""
67	`    if database.count_admins() == 0:`	Condition : branche si la condition est vraie.

L.	Code	Explication
68	`        database.create_admin(`	Appel à la couche base de données.
69	`            DEFAULT_ADMIN_USER, generate_password_hash(DEFAULT_ADMIN_PASS)`	Instruction Python.
70	`        )`	Instruction Python.
71	`        print(f"[!] Admin par défaut créé : {DEFAULT_ADMIN_USER} / {DEFAULT_ADMIN_PASS}")`	Instruction Python.
72	`        print("[!] Change ce mot de passe depuis la page Compte !")`	Instruction Python.

L.	Code	Explication
73		Ligne vide (séparation visuelle).
74		Ligne vide (séparation visuelle).
75	`def login_required(view):`	Déclaration de la fonction login_required().
76	`    @wraps(view)`	Décorateur : wraps(view)
77	`    def wrapped(*args, **kwargs):`	Déclaration de la fonction wrapped().
78	`        if not session.get("admin"):`	Condition : branche si la condition est vraie.
79	`            return redirect(url_for("login"))`	Retourne une valeur au appelant.
80	`        return view(*args, **kwargs)`	Retourne une valeur au appelant.
81	`    return wrapped`	Retourne une valeur au appelant.
82		Ligne vide (séparation visuelle).
83		Ligne vide (séparation visuelle).
84	`def _audit(action, target=None, details=None):`	Déclaration de la fonction _audit().
85	`    """Trace une action administrateur dans le journal d'audit."""`	Commentaire / docstring : """Trace une action administrateur dans le journal d'audit."""
86	`    database.add_audit(session.get("admin"), action, target, details)`	Lit/écrit la session Flask (utilisateur connecté).
87		Ligne vide (séparation visuelle).
88		Ligne vide (séparation visuelle).
89	`# --- Authentification ----------------------------------------------------------------------`	Commentaire / docstring : --- Authentification ----------
90		Ligne vide (séparation visuelle).

L.	Code	Explication
91	`@app.route("/login", methods=["GET", "POST"])`	Décorateur Flask : enregistre une route HTTP.
92	`@limiter.limit("10 per minute", methods=["POST"])`	Limite le nombre de requêtes (anti-abus / brute-force).
93	`def login():`	Déclaration de la fonction login().
94	`    if request.method == "POST":`	Condition : branche si la condition est vraie.
95	`        username = (request.form.get("username") or "").strip()`	Affectation de variable ou constante.
96	`        password = request.form.get("password") or ""`	Affectation de variable ou constante.
97	`        admin = database.get_admin(username)`	Appel à la couche base de données.
98	`        if admin and check_password_hash(admin["password_hash"], password):`	Condition : branche si la condition est vraie.
99	`            if admin.get("totp_secret"):`	Condition : branche si la condition est vraie.
100	`                session["pending_2fa"] = username`	Lit/écrit la session Flask (utilisateur connecté).
101	`                return redirect(url_for("login_2fa"))`	Retourne une valeur au appelant.
102	`            session["admin"] = username`	Lit/écrit la session Flask (utilisateur connecté).
103	`            database.add_audit(username, "Connexion réussie")`	Appel à la couche base de données.
104	`            return redirect(url_for("dashboard"))`	Retourne une valeur au appelant.
105	`        return render_template("login.html", erreur="Identifiants incorrects"), 401`	Retourne une valeur au appelant.
106	`    return render_template("login.html", erreur=None)`	Retourne une valeur au appelant.
107		Ligne vide (séparation visuelle).
108		Ligne vide (séparation visuelle).

L.	Code	Explication
109	`@app.route("/login/2fa", methods=["GET", "POST"])`	Décorateur Flask : enregistre une route HTTP.

L.	Code	Explication
110	`@limiter.limit("10 per minute", methods=["POST"])`	Limite le nombre de requêtes (anti-abus / brute-force).
111	`def login_2fa():`	Déclaration de la fonction login_2fa().
112	`    username = session.get("pending_2fa")`	Lit/écrit la session Flask (utilisateur connecté).
113	`    if not username:`	Condition : branche si la condition est vraie.
114	`        return redirect(url_for("login"))`	Retourne une valeur au appelant.
115	`    if request.method == "POST":`	Condition : branche si la condition est vraie.
116	`        code = (request.form.get("code") or "").strip()`	Affectation de variable ou constante.
117	`        admin = database.get_admin(username)`	Appel à la couche base de données.
118	`        secret = admin.get("totp_secret") if admin else None`	Affectation de variable ou constante.
119	`        if secret and pyotp.TOTP(secret).verify(code, valid_window=1):`	Condition : branche si la condition est vraie.
120	`            session.pop("pending_2fa", None)`	Instruction Python.
121	`            session["admin"] = username`	Lit/écrit la session Flask (utilisateur connecté).
122	`            database.add_audit(username, "Connexion réussie (2FA)")`	Appel à la couche base de données.
123	`            return redirect(url_for("dashboard"))`	Retourne une valeur au appelant.
124	`        return render_template("login_2fa.html", erreur="Code incorrect ou expiré"), 401`	Retourne une valeur au appelant.
125	`    return render_template("login_2fa.html", erreur=None)`	Retourne une valeur au appelant.
126		Ligne vide (séparation visuelle).

L.	Code	Explication
127		Ligne vide (séparation visuelle).
128	`@app.route("/logout")`	Décorateur Flask : enregistre une route HTTP.
129	`def logout():`	Déclaration de la fonction logout().
130	`    _audit("Déconnexion")`	Instruction Python.
131	`    session.clear()`	Instruction Python.
132	`    return redirect(url_for("login"))`	Retourne une valeur au appelant.
133		Ligne vide (séparation visuelle).
134		Ligne vide (séparation visuelle).
135	`@app.route("/account", methods=["GET", "POST"])`	Décorateur Flask : enregistre une route HTTP.
136	`@login_required`	Protège la route : admin connecté requis.
137	`def account():`	Déclaration de la fonction account().
138	`    username = session["admin"]`	Lit/écrit la session Flask (utilisateur connecté).
139	`    tfa = bool((database.get_admin(username) or {}).get("totp_secret"))`	Appel à la couche base de données.
140	`    if request.method == "POST":`	Condition : branche si la condition est vraie.
141	`        actuel = request.form.get("actuel") or ""`	Affectation de variable ou constante.
142	`        nouveau = request.form.get("nouveau") or ""`	Affectation de variable ou constante.
143	`        admin = database.get_admin(username)`	Appel à la couche base de données.
144	`        if not check_password_hash(admin["password_hash"], actuel):`	Condition : branche si la condition est vraie.

L.	Code	Explication
145	`            return render_template("account.html", username=username, tfa=tfa,`	Retourne une valeur au appelant.
146	`                                   erreur="Mot de passe actuel incorrect")`	Affectation de variable ou constante.
147	`        if len(nouveau) < 6:`	Condition : branche si la condition est vraie.
148	`            return render_template("account.html", username=username, tfa=tfa,`	Retourne une valeur au appelant.
149	`                                   erreur="Le nouveau mot de passe doit faire au moins 6 caract…`	Affectation de variable ou constante.
150	`        database.update_admin_password(username, generate_password_hash(nouveau))`	Appel à la couche base de données.
151	`        _audit("Changement de son mot de passe")`	Instruction Python.
152	`        return render_template("account.html", username=username, tfa=tfa, ok=True)`	Retourne une valeur au appelant.

L.	Code	Explication
153	erreur=None, ok=True)	Affectation de variable ou constante.
154	return render_template("account.html", username=username, totp_active=True,	Retourne une valeur au appelant.
155	erreur=None, ok=False)	Affectation de variable ou constante.
156		Ligne vide (séparation visuelle).
157		Ligne vide (séparation visuelle).
158	def _qr_svg(data):	Déclaration de la fonction _qr_svg().
159	"""Génère un QR code SVG (inline, sans dépendance image)."""	Commentaire / docstring : """Génère un QR code SVG (inline, sans dépendance image)."""
160	import io	Import de module ou symbole Python.
161	import qrcode	Import de module ou symbole Python.
162	import qrcode.image.svg	Import de module ou symbole Python.

L.	Code	Explication
163	img = qrcode.make(data, image_factory=qrcode.image.svg.SvgPathImage)	Affectation de variable ou constante.
164	buf = io.BytesIO()	Affectation de variable ou constante.
165	img.save(buf)	Instruction Python.
166	return buf.getvalue().decode("utf-8")	Retourne une valeur au appelant.
167		Ligne vide (séparation visuelle).
168		Ligne vide (séparation visuelle).
169	@app.route("/account/2fa/setup")	Décorateur Flask : enregistre une route HTTP.
170	@login_required	Protège la route : admin connecté requis.
171	def twofa_setup():	Déclaration de la fonction twofa_setup().
172	username = session["admin"]	Lit/écrit la session Flask (utilisateur connecté).
173	if (database.get_admin(username) or {}).get("totp_secret"):	Condition : branche si la condition est vraie.
174	return redirect(url_for("account"))	Retourne une valeur au appelant.
175	secret = session.get("twofa_tmp") or pyotp.random_base32()	Lit/écrit la session Flask (utilisateur connecté).
176	session["twofa_tmp"] = secret	Lit/écrit la session Flask (utilisateur connecté).
177	uri = pyotp.TOTP(secret).provisioning_uri(name=username, issuer_name="NomCourt")	Affectation de variable ou constante.
178	return render_template("twofa_setup.html", secret=secret, qr_svg=_qr_svg(uri), erreur=None)	Retourne une valeur au appelant.
179		Ligne vide (séparation visuelle).
180		Ligne vide (séparation visuelle).

L.	Code	Explication
181	@app.route("/account/2fa/enable", methods=["POST"])	Décorateur Flask : enregistre une route HTTP.
182	@login_required	Protège la route : admin connecté requis.
183	def twofa_enable():	Déclaration de la fonction twofa_enable().
184	username = session["admin"]	Lit/écrit la session Flask (utilisateur connecté).
185	secret = session.get("twofa_tmp")	Lit/écrit la session Flask (utilisateur connecté).
186	code = (request.form.get("code") or "").strip()	Affectation de variable ou constante.
187	if not secret:	Condition : branche si la condition est vraie.
188	return redirect(url_for("twofa_setup"))	Retourne une valeur au appelant.
189	if not pyotp.TOTP(secret).verify(code, valid_window=1):	Condition : branche si la condition est vraie.
190	uri = pyotp.TOTP(secret).provisioning_uri(name=username, issuer_name="NomCourt")	Affectation de variable ou constante.
191	return render_template("twofa_setup.html", secret=secret, qr_svg=_qr_svg(uri),	Retourne une valeur au appelant.
192	erreur="Code incorrect, réessaie."), 400	Affectation de variable ou constante.
193	database.set_admin_totp(username, secret)	Appel à la couche base de données.
194	session.pop("twofa_tmp", None)	Instruction Python.
195	_audit("Activation de la 2FA")	Instruction Python.

L.	Code	Explication
196	`    return redirect(url_for("account"))`	Retourne une valeur au appelant.
197		Ligne vide (séparation visuelle).
198		Ligne vide (séparation visuelle).

L.	Code	Explication
199	`@app.route("/account/2fa/disable", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
200	`@login_required`	Protège la route : admin connecté requis.
201	`def twofa_disable():`	Déclaration de la fonction twofa_disable().
202	`    database.set_admin_totp(session["admin"], None)`	Lit/écrit la session Flask (utilisateur connecté).
203	`    _audit("Désactivation de la 2FA")`	Instruction Python.
204	`    return redirect(url_for("account"))`	Retourne une valeur au appelant.
205		Ligne vide (séparation visuelle).
206		Ligne vide (séparation visuelle).
207	`# --- Dashboard ----------------------------------------`	Commentaire / docstring : --- Dashboard ----------
208		Ligne vide (séparation visuelle).
209	`@app.route("/")`	Décorateur Flask : enregistre une route HTTP.
210	`@login_required`	Protège la route : admin connecté requis.
211	`def dashboard():`	Déclaration de la fonction dashboard().
212	`    admin = database.get_admin(session["admin"])`	Lit/écrit la session Flask (utilisateur connecté).
213	`    mdp_par_defaut = check_password_hash(admin["password_hash"], DEFAULT_ADMIN_PASS)`	Affectation de variable ou constante.
214	`    return render_template("dashboard.html", mdp_par_defaut=mdp_par_defaut)`	Retourne une valeur au appelant.
215		Ligne vide (séparation visuelle).
216		Ligne vide (séparation visuelle).

L.	Code	Explication
217	`# Messages frauduleux types pour la démonstration en soutenance.`	Commentaire / docstring : Messages frauduleux types pour la démonstration en soutenance.
218	`DEMO_SAMPLES = [`	Affectation de variable ou constante.
219	`    "URGENT: votre compte Orange Money sera suspendu. Reactivez sur http://0range-money.tk",`	Instruction Python.
220	`    "M-Pesa: votre carte SIM va etre remplacee. Communiquez votre code PUK pour confirmer.",`	Instruction Python.
221	`    "Felicitations ! Vous avez gagne 5.000.000 FC, envoyez 10$ de frais pour recevoir.",`	Instruction Python.
222	`    "Bonjour, je suis agent Airtel Money. Donnez le code de validation recu pour securiser votr…`	Instruction Python.
223	`    "Securite Vodacom: activite suspecte. Confirmez votre identite sur http://v0dacom-secure.xy…`	Instruction Python.
224	`]`	Instruction Python.
225		Ligne vide (séparation visuelle).
226		Ligne vide (séparation visuelle).
227	`@app.route("/demo", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
228	`@login_required`	Protège la route : admin connecté requis.
229	`def demo_alert():`	Déclaration de la fonction demo_alert().
230	`    """Génère une alerte frauduleuse de démonstration (utile en soutenance)."""`	Commentaire / docstring : """Génère une alerte frauduleuse de démonstration (utile en soutenance)."""
231	`    import random`	Import de module ou symbole Python.
232	`    content = random.choice(DEMO_SAMPLES)`	Affectation de variable ou constante.
233	`    score, level, reasons = scoring.score_event(`	Appel au moteur de scoring.
234	`        "sms", content, {}, database.get_thresholds()`	Appel à la couche base de données.

L.	Code	Explication
235	`    )`	Instruction Python.
236	`    saved = database.insert_report(`	Appel à la couche base de données.
237	`        "demo-jury", "demo", "sms", content, {}, score, level, reasons`	Affectation de variable ou constante.

L.	Code	Explication
238	`    )`	Instruction Python.
239	`    socketio.emit("new_report", saved)`	Émet un événement WebSocket vers les dashboards connectés.
240	`    _audit("Alerte de démonstration générée")`	Instruction Python.
241	`    return jsonify({"ok": True, "level": level, "id": saved["id"]})`	Retourne une valeur au appelant.
242		Ligne vide (séparation visuelle).
243		Ligne vide (séparation visuelle).
244	`@app.route("/api/reports")`	Décorateur Flask : enregistre une route HTTP.
245	`@login_required`	Protège la route : admin connecté requis.
246	`def api_reports():`	Déclaration de la fonction api_reports().
247	`    return jsonify({`	Retourne une valeur au appelant.
248	`        "reports": database.get_recent(limit=100),`	Appel à la couche base de données.
249	`        "stats": database.get_stats(),`	Appel à la couche base de données.
250	`    })`	Instruction Python.
251		Ligne vide (séparation visuelle).
252		Ligne vide (séparation visuelle).

L.	Code	Explication
253	`# --- Réception des rapports (agents) ----------`	Commentaire / docstring : --- Réception des rapports (agents) ----------
254		Ligne vide (séparation visuelle).
255	`@app.route("/report", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
256	`@csrf.exempt`	Exempte la route de la protection CSRF (API JSON agents).
257	`@limiter.limit("120 per minute")`	Limite le nombre de requêtes (anti-abus / brute-force).
258	`def report():`	Déclaration de la fonction report().
259	`    data = request.get_json(silent=True)`	Affectation de variable ou constante.
260	`    if not data:`	Condition : branche si la condition est vraie.
261	`        return jsonify({"error": "Corps JSON manquant ou invalide"}), 400`	Retourne une valeur au appelant.
262		Ligne vide (séparation visuelle).
263	`    agent_id = data.get("agent_id", "inconnu")`	Affectation de variable ou constante.
264	`    platform = data.get("platform", "inconnu")`	Affectation de variable ou constante.
265	`    event_type = data.get("event_type", "message")`	Affectation de variable ou constante.
266	`    content = data.get("content", "")`	Affectation de variable ou constante.
267	`    metadata = data.get("metadata", {})`	Affectation de variable ou constante.
268	`    token = data.get("token")`	Affectation de variable ou constante.
269	`    uid = data.get("uid")`	Affectation de variable ou constante.
270		Ligne vide (séparation visuelle).

L.	Code	Explication
271	`    if not content:`	Condition : branche si la condition est vraie.
272	`        return jsonify({"error": "Le champ 'content' est requis"}), 400`	Retourne une valeur au appelant.
273		Ligne vide (séparation visuelle).
274	`    # Contrôle d'adhésion : si l'agent est rattaché à un compte client,`	Commentaire / docstring : Contrôle d'adhésion : si l'agent est rattaché à un compte client,
275	`    # ce compte doit avoir été approuvé par l'admin.`	Commentaire / docstring : ce compte doit avoir été approuvé par l'admin.
276	`    user = database.get_user_by_uid(uid) if uid else None`	Appel à la couche base de données.
277	`    if user and user["status"] != "approved":`	Condition : branche si la condition est vraie.
278	`        libelle = {`	Affectation de variable ou constante.
279	`            "pending": "Compte en attente d'approbation",`	Instruction Python.
280	`            "rejected": "Compte refusé",`	Instruction Python.

L.	Code	Explication
281	`            "blocked": "Compte bloqué",`	Instruction Python.
282	`        }.get(user["status"], "Compte non approuvé")`	Instruction Python.
283	`        return jsonify({"error": libelle}), 403`	Retourne une valeur au appelant.
284		Ligne vide (séparation visuelle).
285	`    # Contrôle d'accès de l'agent`	Commentaire / docstring : Contrôle d'accès de l'agent
286	`    agent = database.get_agent(agent_id)`	Appel à la couche base de données.
287	`    if agent:`	Condition : branche si la condition est vraie.
288	`        if agent["status"] == "blocked":`	Condition : branche si la condition est vraie.

L.	Code	Explication
289	`            return jsonify({"error": "Agent bloqué"}), 403`	Retourne une valeur au appelant.
290	`        if agent["token"] and token != agent["token"]:`	Condition : branche si la condition est vraie.
291	`            return jsonify({"error": "Jeton d'agent invalide"}), 401`	Retourne une valeur au appelant.
292	`        database.touch_agent(agent_id)`	Appel à la couche base de données.
293	`    else:`	Branche alternative (sinon).
294	`        # Agent inconnu : on l'enregistre automatiquement pour qu'il soit gérable`	Commentaire / docstring : Agent inconnu : on l'enregistre automatiquement pour qu'il soit gérable
295	`        database.register_agent(agent_id, "(auto)", platform, None)`	Appel à la couche base de données.
296	`        database.touch_agent(agent_id)`	Appel à la couche base de données.
297		Ligne vide (séparation visuelle).
298	`    score, level, reasons = scoring.score_event(`	Appel au moteur de scoring.
299	`        event_type, content, metadata, database.get_thresholds()`	Appel à la couche base de données.
300	`    )`	Instruction Python.
301		Ligne vide (séparation visuelle).
302	`    # Enrichissement par réputation d'URL via une API externe (si configurée)`	Commentaire / docstring : Enrichissement par réputation d'URL via une API externe (si configurée).
303	`    if threat_intel.is_ready():`	Condition : branche si la condition est vraie.
304	`        urls = threat_intel.extract_urls(content)`	Appel au module threat intelligence.
305	`        if urls and threat_intel.check_urls(urls) == "malicious":`	Condition : branche si la condition est vraie.
306	`            score = min(score + 40, 100)`	Affectation de variable ou constante.

L.	Code	Explication
307	`            level = "eleve"`	Affectation de variable ou constante.
308	`            reasons.append("URL signalée comme malveillante par le service de réputation extern…`	Instruction Python.
309		Ligne vide (séparation visuelle).
310	`    saved = database.insert_report(`	Appel à la couche base de données.
311	`        agent_id, platform, event_type, content, metadata, score, level, reasons,`	Instruction Python.
312	`        uid=uid,`	Affectation de variable ou constante.
313	`    )`	Instruction Python.
314	`    socketio.emit("new_report", saved)`	Émet un événement WebSocket vers les dashboards connectés.
315		Ligne vide (séparation visuelle).
316	`    # Notification push de danger au client concerné (alerte de niveau élevé)`	Commentaire / docstring : Notification push de danger au client concerné (alerte de niveau élevé).
317	`    if level == "eleve" and user and user.get("fcm_token"):`	Condition : branche si la condition est vraie.
318	`        firebase_auth.send_push(`	Appel au module Firebase.
319	`            user["fcm_token"],`	Instruction Python.
320	`            "MoMoGuard — danger détecté",`	Instruction Python.
321	`            "Un message à haut risque a été repéré sur ton appareil. Vérifie tes alertes.",`	Instruction Python.
322	`        )`	Instruction Python.
323		Ligne vide (séparation visuelle).

L.	Code	Explication
324	`    return jsonify({`	Retourne une valeur au appelant.
325	`        "ok": True, "id": saved["id"], "score": score,`	Instruction Python.
326	`        "level": level, "reasons": reasons,`	Instruction Python.
327	`    }), 201`	Instruction Python.
328	``	Ligne vide (séparation visuelle).
329	``	Ligne vide (séparation visuelle).
330	`# --- Gestion des agents ----------------------

L.	Code	Explication
366		Ligne vide (séparation visuelle).
367		Ligne vide (séparation visuelle).
368	`@app.route("/agents/<agent_id>/block", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
369	`@login_required`	Protège la route : admin connecté requis.
370	`def agent_block(agent_id):`	Déclaration de la fonction agent_block().
371	`    database.set_agent_status(agent_id, "blocked")`	Appel à la couche base de données.
372	`    _audit("Agent bloqué", target=agent_id)`	Affectation de variable ou constante.
373	`    return redirect(url_for("agents"))`	Retourne une valeur au appelant.
374		Ligne vide (séparation visuelle).
375		Ligne vide (séparation visuelle).
376	`@app.route("/agents/<agent_id>/unblock", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
377	`@login_required`	Protège la route : admin connecté requis.
378	`def agent_unblock(agent_id):`	Déclaration de la fonction agent_unblock().

L.	Code	Explication
379	`    database.set_agent_status(agent_id, "active")`	Appel à la couche base de données.
380	`    _audit("Agent débloqué", target=agent_id)`	Affectation de variable ou constante.
381	`    return redirect(url_for("agents"))`	Retourne une valeur au appelant.
382		Ligne vide (séparation visuelle).
383		Ligne vide (séparation visuelle).
384	`@app.route("/agents/<agent_id>/remove", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
385	`@login_required`	Protège la route : admin connecté requis.
386	`def agent_remove(agent_id):`	Déclaration de la fonction agent_remove().
387	`    database.delete_agent(agent_id)`	Appel à la couche base de données.
388	`    _audit("Agent retiré", target=agent_id)`	Affectation de variable ou constante.
389	`    return redirect(url_for("agents"))`	Retourne une valeur au appelant.
390		Ligne vide (séparation visuelle).
391		Ligne vide (séparation visuelle).
392	`# --- Authentification client (Firebase) ----------`	Commentaire / docstring : --- Authentification client (Firebase) ----------
393		Ligne vide (séparation visuelle).
394	`@app.route("/api/auth/firebase", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
395	`@csrf.exempt`	Exempte la route de la protection CSRF (API JSON agents).
396	`def auth_firebase():`	Déclaration de la fonction auth_firebase().

L.	Code	Explication
397	`    """L'app envoie son ID token Firebase. On le vérifie, puis on crée/relie`	Commentaire / docstring : """L'app envoie son ID token Firebase. On le vérifie, puis on crée/relie
398	`    l'utilisateur dans MySQL (statut 'pending' tant que l'admin n'a pas approuvé)."""`	l'utilisateur dans MySQL (statut 'pending' tant que l'admin n'a pas approuvé).""" Condition : branche si la condition est vraie.
399	`    if not firebase_auth.is_ready():`	
400	`        return jsonify({"error": "Firebase non configuré sur le serveur"}), 503`	Retourne une valeur au appelant.
401		Ligne vide (séparation visuelle).
402	`    data = request.get_json(silent=True) or {}`	Affectation de variable ou constante.
403	`    id_token = data.get("id_token")`	Affectation de variable ou constante.
404	`    if not id_token:`	Condition : branche si la condition est vraie.
405	`        return jsonify({"error": "id_token manquant"}), 400`	Retourne une valeur au appelant.
406		Ligne vide (séparation visuelle).
407	`    try:`	Début bloc try/except (gestion d'erreurs).
408	`        decoded = firebase_auth.verify_id_token(id_token)`	Appel au module Firebase.

L.	Code	Explication
409	`    except Exception:`	Capture une exception.
410	`        return jsonify({"error": "Jeton Firebase invalide ou expiré"}), 401`	Retourne une valeur au appelant.
411		Ligne vide (séparation visuelle).
412	`    uid = decoded.get("uid")`	Affectation de variable ou constante.
413	`    email = decoded.get("email", "")`	Affectation de variable ou constante.
414	`    name = decoded.get("name", "")`	Affectation de variable ou constante.

L.	Code	Explication
415	`    # IP du client (gère le cas d'un proxy via X-Forwarded-For)`	Commentaire / docstring : IP du client (gère le cas d'un proxy via X-Forwarded-For)
416	`    ip = (request.headers.get("X-Forwarded-For", "").split(",")[0].strip()`	Affectation de variable ou constante.
417	`          or request.remote_addr)`	Instruction Python.
418	`    user = database.upsert_user(uid, email, name, ip=ip)`	Appel à la couche base de données.
419		Ligne vide (séparation visuelle).
420	`    return jsonify({`	Retourne une valeur au appelant.
421	`        "ok": True,`	Instruction Python.
422	`        "uid": user["uid"],`	Instruction Python.
423	`        "email": user["email"],`	Instruction Python.
424	`        "status": user["status"],`	Instruction Python.
425	`    })`	Instruction Python.
426		Ligne vide (séparation visuelle).
427		Ligne vide (séparation visuelle).
428	`@app.route("/api/my-reports", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
429	`@csrf.exempt`	Exempte la route de la protection CSRF (API JSON agents).
430	`def my_reports():`	Déclaration de la fonction my_reports().
431	`    """L'app envoie son ID token ; on renvoie les alertes de CE client."""`	Commentaire / docstring : """L'app envoie son ID token ; on renvoie les alertes de CE client."""
432	`    if not firebase_auth.is_ready():`	Condition : branche si la condition est vraie.

L.	Code	Explication
433	`        return jsonify({"error": "Firebase non configuré sur le serveur"}), 503`	Retourne une valeur au appelant.
434	`    data = request.get_json(silent=True) or {}`	Affectation de variable ou constante.
435	`    id_token = data.get("id_token")`	Affectation de variable ou constante.
436	`    if not id_token:`	Condition : branche si la condition est vraie.
437	`        return jsonify({"error": "id_token manquant"}), 400`	Retourne une valeur au appelant.
438	`    try:`	Début bloc try/except (gestion d'erreurs).
439	`        decoded = firebase_auth.verify_id_token(id_token)`	Appel au module Firebase.
440	`    except Exception:`	Capture une exception.
441	`        return jsonify({"error": "Jeton Firebase invalide ou expiré"}), 401`	Retourne une valeur au appelant.
442		Ligne vide (séparation visuelle).
443	`    uid = decoded.get("uid")`	Affectation de variable ou constante.
444	`    user = database.get_user_by_uid(uid)`	Appel à la couche base de données.
445	`    reports = database.get_reports_for_user(uid, limit=100)`	Appel à la couche base de données.
446	`    counts = {"total": len(reports), "faible": 0, "moyen": 0, "fort": 0}`	Affectation de variable ou constante.
447	`    for r in reports:`	Boucle de répétition.
448	`        counts[r["level"]] = counts.get(r["level"], 0) + 1`	Affectation de variable ou constante.
449	`    return jsonify({`	Retourne une valeur au appelant.
450	`        "status": user["status"] if user else "unknown",`	Instruction Python.

L.	Code	Explication
451	`        "stats": counts,`	Instruction Python.
452	`        "reports": reports,`	Instruction Python.
453	`    })`	Instruction Python.
454		Ligne vide (séparation visuelle).
455		Ligne vide (séparation visuelle).
456	`@app.route("/api/fcm-token", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
457	`@csrf.exempt`	Exempte la route de la protection CSRF (API JSON agents).
458	`def fcm_token():`	Déclaration de la fonction fcm_token().
459	`    """L'app enregistre (ou met à jour) son jeton FCM pour recevoir les push."""`	Commentaire / docstring : """L'app enregistre (ou met à jour) son jeton FCM pour recevoir les push."""
460	`    if not firebase_auth.is_ready():`	Condition : branche si la condition est vraie.
461	`        return jsonify({"error": "Firebase non configuré sur le serveur"}), 503`	Retourne une valeur au appelant.
462	`    data = request.get_json(silent=True) or {}`	Affectation de variable ou constante.
463	`    id_token = data.get("id_token")`	Affectation de variable ou constante.
464	`    fcm = data.get("fcm_token")`	Affectation de variable ou constante.
465	`    if not id_token or not fcm:`	Condition : branche si la condition est vraie.
466	`        return jsonify({"error": "id_token et fcm_token requis"}), 400`	Retourne une valeur au appelant.
467	`    try:`	Début bloc try/except (gestion d'erreurs).
468	`        decoded = firebase_auth.verify_id_token(id_token)`	Appel au module Firebase.

L.	Code	Explication
469	`    except Exception:`	Capture une exception.
470	`        return jsonify({"error": "Jeton Firebase invalide ou expiré"}), 401`	Retourne une valeur au appelant.
471	`    database.set_user_fcm_token(decoded.get("uid"), fcm)`	Appel à la couche base de données.
472	`    return jsonify({"ok": True})`	Retourne une valeur au appelant.
473		Ligne vide (séparation visuelle).
474		Ligne vide (séparation visuelle).
475	`# --- Gestion des utilisateurs clients ----------`	Commentaire / docstring : --- Gestion des utilisateurs clients ----------
476		Ligne vide (séparation visuelle).
477	`@app.route("/users")`	Décorateur Flask : enregistre une route HTTP.
478	`@login_required`	Protège la route : admin connecté requis.
479	`def users():`	Déclaration de la fonction users().
480	`    return render_template("users.html", users=database.get_users())`	Retourne une valeur au appelant.
481		Ligne vide (séparation visuelle).
482		Ligne vide (séparation visuelle).
483	`@app.route("/users/<uid>/approve", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
484	`@login_required`	Protège la route : admin connecté requis.
485	`def user_approve(uid):`	Déclaration de la fonction user_approve().
486	`    database.set_user_status(uid, "approved")`	Appel à la couche base de données.

L.	Code	Explication
487	`    _audit("Utilisateur approuvé", target=uid)`	Affectation de variable ou constante.
488	`    return redirect(url_for("users"))`	Retourne une valeur au appelant.
489		Ligne vide (séparation visuelle).
490		Ligne vide (séparation visuelle).
491	`@app.route("/users/<uid>/reject", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
492	`@login_required`	Protège la route : admin connecté requis.
493	`def user_reject(uid):`	Déclaration de la fonction user_reject().

L.	Code	Explication
494	`    database.set_user_status(uid, "rejected")`	Appel à la couche base de données.
495	`    _audit("Utilisateur refusé", target=uid)`	Affectation de variable ou constante.
496	`    return redirect(url_for("users"))`	Retourne une valeur au appelant.
497		Ligne vide (séparation visuelle).
498		Ligne vide (séparation visuelle).
499	`@app.route("/users/<uid>/block", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
500	`@login_required`	Protège la route : admin connecté requis.
501	`def user_block(uid):`	Déclaration de la fonction user_block().
502	`    database.set_user_status(uid, "blocked")`	Appel à la couche base de données.
503	`    _audit("Utilisateur bloqué", target=uid)`	Affectation de variable ou constante.
504	`    return redirect(url_for("users"))`	Retourne une valeur au appelant.

L.	Code	Explication
505		Ligne vide (séparation visuelle).
506		Ligne vide (séparation visuelle).
507	`@app.route("/users/<uid>")`	Décorateur Flask : enregistre une route HTTP.
508	`@login_required`	Protège la route : admin connecté requis.
509	`def user_detail(uid):`	Déclaration de la fonction user_detail().
510	`    user = database.get_user_by_uid(uid)`	Appel à la couche base de données.
511	`    if not user:`	Condition : branche si la condition est vraie.
512	`        return redirect(url_for("users"))`	Retourne une valeur au appelant.
513	`    return render_template("user_detail.html", user=user, erreur=None, message=None)`	Retourne une valeur au appelant.
514		Ligne vide (séparation visuelle).
515		Ligne vide (séparation visuelle).
516	`@app.route("/users/<uid>/edit", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
517	`@login_required`	Protège la route : admin connecté requis.
518	`def user_edit(uid):`	Déclaration de la fonction user_edit().
519	`    name = (request.form.get("display_name") or "").strip()`	Affectation de variable ou constante.
520	`    database.update_user(uid, name)`	Appel à la couche base de données.
521	`    _audit("Modification utilisateur", target=uid, detail=f"nom → {name}")`	Affectation de variable ou constante.
522	`    return redirect(url_for("user_detail", uid=uid))`	Retourne une valeur au appelant.

L.	Code	Explication
523		Ligne vide (séparation visuelle).
524		Ligne vide (séparation visuelle).
525	`@app.route("/users/<uid>/password", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
526	`@login_required`	Protège la route : admin connecté requis.
527	`def user_password(uid):`	Déclaration de la fonction user_password().
528	`    user = database.get_user_by_uid(uid)`	Appel à la couche base de données.
529	`    if not user:`	Condition : branche si la condition est vraie.
530	`        return redirect(url_for("users"))`	Retourne une valeur au appelant.
531	`    new_pwd = request.form.get("new_password") or ""`	Affectation de variable ou constante.
532	`    if len(new_pwd) < 6:`	Condition : branche si la condition est vraie.
533	`        return render_template("user_detail.html", user=user, erreur=None, message=None)`	Retourne une valeur au appelant.
534	`                               erreur="Le mot de passe doit faire au moins 6 caractères."), 400`	Affectation de variable ou constante.
535	`    if not firebase_auth.is_ready():`	Condition : branche si la condition est vraie.
536	`        return render_template("user_detail.html", user=user, erreur=None, message=None)`	Retourne une valeur au appelant.

L.	Code	Explication
537	erreur="Firebase non configuré côté serveur."), 503	Affectation de variable ou constante.
538	try:	Début bloc try/except (gestion d'erreurs).
539	firebase_auth.update_user_password(uid, new_pwd)	Appel au module Firebase.
540	except Exception as e:	Capture une exception.

L.	Code	Explication
541	return render_template("user_detail.html", user=user, ok=False, msg=None,	Retourne la réponse au navigateur.
542	erreur=f"Échec de la mise à jour : {e}"), 500	Affectation de variable ou constante.
543	notified = False	Affectation de variable ou constante.
544	if user.get("fcm_token"):	Condition : branche si la condition est vraie.
545	notified = firebase_auth.send_push(	Appel au module Firebase.
546	user["fcm_token"], "MoMoGuard — sécurité",	Instruction Python.
547	"Votre mot de passe a été modifié par l'administrateur.",	Instruction Python.
548	"Si ce n'est pas vous, contactez le support.",	Instruction Python.
549	)	Instruction Python.
550	msg = ("Mot de passe mis à jour. "	Affectation de variable ou constante.
551	+ ("L'utilisateur a été notifié." if notified	Instruction Python.
552	else "(Pas de notification : aucun jeton FCM enregistré pour ce compte.)"))	Instruction Python.
553	_audit("Changement mot de passe utilisateur", target=uid,	Affectation de variable ou constante.
554	details=("notifié" if notified else "non notifié"))	Affectation de variable ou constante.
555	return render_template("user_detail.html", user=database.get_user_by_uid(uid),	Retourne la réponse au navigateur.
556	ok=True, msg=msg, erreur=None)	Affectation de variable ou constante.
557		Ligne vide (séparation visuelle).
558		Ligne vide (séparation visuelle).

L.	Code	Explication
559	# --- Paramètres (seuils de score) ----------	Commentaire / docstring : --- Paramètres (seuils de score) ----------
560		Ligne vide (séparation visuelle).
561	@app.route("/settings", methods=["GET", "POST"])	Décorateur Flask : enregistre une route HTTP.
562	@login_required	Protège la route : admin connecté requis.
563	def settings_page():	Déclaration de la fonction settings_page().
564	if request.method == "POST":	Condition : branche si la condition est vraie.
565	try:	Début bloc try/except (gestion d'erreurs).
566	moyen = int(request.form.get("seuil_moyen", 25))	Affectation de variable ou constante.
567	eleve = int(request.form.get("seuil_eleve", 50))	Affectation de variable ou constante.
568	except ValueError:	Capture une exception.
569	m, e = database.get_thresholds()	Appel à la couche base de données.
570	return render_template("settings.html", moyen=m, eleve=e,	Retourne la réponse au navigateur.
571	erreur="Valeurs invalides.", ok=False), 400	Affectation de variable ou constante.
572	if not (0 <= moyen < eleve <= 100):	Condition : branche si la condition est vraie.
573	m, e = database.get_thresholds()	Appel à la couche base de données.
574	return render_template("settings.html", moyen=m, eleve=e,	Retourne la réponse au navigateur.
575	erreur="Il faut 0 ≤ moyen < élevé ≤ 100.", ok=False), 400	Affectation de variable ou constante.
576	database.set_thresholds(moyen, eleve)	Appel à la couche base de données.

L.	Code	Explication
577	_audit("Modification des seuils", details=f"{moyen}/{eleve}")	Affectation de variable ou constante.
578	return render_template("settings.html", moyen=moyen, eleve=eleve, msg="Seuils enregistrés.", ok=True)	Retourne la réponse au navigateur.

L.	Code	Explication
579	`    m, e = database.get_thresholds()`	Appel à la couche base de données.
580	`    return render_template("settings.html", moyen=m, eleve=e, erreur=None, saved=False)`	Retourne une valeur au appelant.
581		Ligne vide (séparation visuelle).
582		Ligne vide (séparation visuelle).
583	`# --- Consentement ----------------------------------------------------------------------`	Commentaire / docstring : --- Consentement ----------
584		Ligne vide (séparation visuelle).
585	`SCOPES = [`	Affectation de variable ou constante.
586	`    ("clipboard", "Presse-papier (numéros, liens copiés)"),`	Instruction Python.
587	`    ("link", "Liens reçus ou ouverts"),`	Instruction Python.
588	`    ("sms", "Messages SMS (Android)"),`	Instruction Python.
589	`    ("file", "Fichiers téléchargés"),`	Instruction Python.
590	`    ("ussd", "Codes USSD composés"),`	Instruction Python.
591	`]`	Instruction Python.
592		Ligne vide (séparation visuelle).
593		Ligne vide (séparation visuelle).
594	`@app.route("/consent", methods=["GET", "POST"])`	Décorateur Flask : enregistre une route HTTP.

L.	Code	Explication
595	`def consent():`	Déclaration de la fonction consent().
596	`    if request.method == "POST":`	Condition : branche si la condition est vraie.
597	`        participant = (request.form.get("participant") or "").strip()`	Affectation de variable ou constante.
598	`        role = (request.form.get("role") or "").strip()`	Affectation de variable ou constante.
599	`        device = (request.form.get("device") or "").strip()`	Affectation de variable ou constante.
600	`        scope = request.form.getlist("scope")`	Affectation de variable ou constante.
601	`        comprend = request.form.get("comprend")`	Affectation de variable ou constante.
602	`        if not participant or not comprend or not scope:`	Condition : branche si la condition est vraie.
603	`            return render_template(`	Retourne une valeur au appelant.
604	`                "consent.html", scopes=SCOPES,`	Affectation de variable ou constante.
605	`                erreur="Renseigne ton nom, coche au moins une case et confirme avoir compri…`	Affectation de variable ou constante.
606	`            ), 400`	Instruction Python.
607	`        database.insert_consent(participant, role, device, scope)`	Appel à la couche base de données.
608	`        return redirect(url_for("consents"))`	Retourne une valeur au appelant.
609	`    return render_template("consent.html", scopes=SCOPES, erreur=None)`	Retourne une valeur au appelant.
610		Ligne vide (séparation visuelle).
611		Ligne vide (séparation visuelle).
612	`@app.route("/consents")`	Décorateur Flask : enregistre une route HTTP.

L.	Code	Explication
613	`@login_required`	Protège la route : admin connecté requis.
614	`def consents():`	Déclaration de la fonction consents().
615	`    return render_template("consents.html", consents=database.get_consents())`	Retourne une valeur au appelant.
616		Ligne vide (séparation visuelle).
617		Ligne vide (séparation visuelle).
618	`@app.route("/consent/<int:cid>/revoke", methods=["POST"])`	Décorateur Flask : enregistre une route HTTP.
619	`@login_required`	Protège la route : admin connecté requis.
620	`def consent_revoke(cid):`	Déclaration de la fonction consent_revoke().
621	`    database.revoke_consent(cid)`	Appel à la couche base de données.

L.	Code	Explication
622	`    _audit("Consentement révoqué", target=str(cid))`	Affectation de variable ou constante.
623	`    return redirect(url_for("consents"))`	Retourne une valeur au appelant.
624		Ligne vide (séparation visuelle).
625		Ligne vide (séparation visuelle).
626	`@app.route("/audit")`	Décorateur Flask : enregistre une route HTTP.
627	`@login_required`	Protège la route : admin connecté requis.
628	`def audit():`	Déclaration de la fonction audit().
629	`    return render_template("audit.html", entries=database.get_audit())`	Retourne une valeur au appelant.
630		Ligne vide (séparation visuelle).

L.	Code	Explication
631		Ligne vide (séparation visuelle).
632	`@socketio.on("connect")`	Écouteur d'événement WebSocket SocketIO.
633	`def on_connect():`	Déclaration de la fonction on_connect().
634	`    print("[+] Un dashboard s'est connecté")`	Instruction Python.
635		Ligne vide (séparation visuelle).
636		Ligne vide (séparation visuelle).
637	`def main():`	Déclaration de la fonction main().
638	`    """Initialise la base, l'admin et démarre le serveur."""`	Commentaire / docstring : """Initialise la base, l'admin et démarre le serveur."""
639	`    database.init_db()`	Appel à la couche base de données.
640	`    ensure_default_admin()`	Instruction Python.
641	`    if firebase_auth.init_firebase():`	Condition : branche si la condition est vraie.
642	`        print("[Firebase] authentification client activée")`	Instruction Python.
643	`    else:`	Branche alternative (sinon).
644	`        print("[Firebase] clé absente — authentification client désactivée")`	Instruction Python.
645	`    print("=" * 60)`	Affectation de variable ou constante.
646	`    print(" MoMoGuard - serveur central")`	Instruction Python.
647	`    print(" Dashboard : http://127.0.0.1:5000  (connexion requise)")`	Instruction Python.
648	`    print(" API       : POST http://127.0.0.1:5000/report")`	Instruction Python.

L.	Code	Explication
649	`    print("=" * 60)`	Affectation de variable ou constante.
650	`    socketio.run(app, host="0.0.0.0", port=5000, debug=True,`	Affectation de variable ou constante.
651	`                 allow_unsafe_werkzeug=True)`	Affectation de variable ou constante.
652		Ligne vide (séparation visuelle).
653		Ligne vide (séparation visuelle).
654	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
655	`    main()`	Instruction Python.

Fichier : server/database.py

Couche SQLAlchemy : tables, requêtes, conseils.

Fonctions déclarées

_make_engine(url) — ligne 110

→ Fonction utilitaire « _make_engine ».
set_database_url(url) — ligne 121

→ Permet de changer de base à chaud (utilisé surtout pour les tests).
init_db() — ligne 128

→ Crée les tables et applique migrations douces.
_ensure_column(table_name, column, coltype) — ligne 136

→ Ajoute une colonne manquante sur une table existante (migration douce).
_now() — ligne 147

→ Fonction utilitaire « _now ».
insert_report(agent_id, platform, event_type, content, metadata, score, level, reasons, uid) — ligne 153

→ Insère un rapport d'alerte en base.
_conseil(reasons) — ligne 193

→ Renvoie un conseil de prévention adapté aux raisons de l'alerte.
_report_to_dict(r) — ligne 202

→ Fonction utilitaire « _report_to_dict ».
get_recent(limit) — ligne 215

→ Récupère les alertes récentes.
get_reports_for_user(uid, limit) — ligne 223

→ Fonction utilitaire « get_reports_for_user ».
get_stats() — ligne 232

→ Fonction utilitaire « get_stats ».
insert_consent(participant, role, device, scope) — ligne 246

→ Fonction utilitaire « insert_consent ».
get_consents() — ligne 260

→ Fonction utilitaire « get_consents ».
revoke_consent(consent_id) — ligne 272

→ Fonction utilitaire « revoke_consent ».
count_admins() — ligne 283

→ Fonction utilitaire « count_admins ».
create_admin(username, password_hash) — ligne 288

→ Fonction utilitaire « create_admin ».
get_admin(username) — ligne 295

→ Fonction utilitaire « get_admin ».
update_admin_password(username, password_hash) — ligne 303

→ Fonction utilitaire « update_admin_password ».
set_admin_totp(username, secret) — ligne 311

→ Active (secret) ou désactive (None) la 2FA pour un administrateur.
_agent_to_dict(r) — ligne 322

→ Fonction utilitaire « _agent_to_dict ».
register_agent(agent_id, label, platform, token) — ligne 330

→ Fonction utilitaire « register_agent ».
get_agent(agent_id) — ligne 341

→ Fonction utilitaire « get_agent ».
get_agents() — ligne 349

→ Fonction utilitaire « get_agents ».
set_agent_status(agent_id, status) — ligne 357

→ Fonction utilitaire « set_agent_status ».
delete_agent(agent_id) — ligne 364

→ Fonction utilitaire « delete_agent ».
touch_agent(agent_id) — ligne 369

→ Fonction utilitaire « touch_agent ».
_user_to_dict(r) — ligne 378

→ Fonction utilitaire « _user_to_dict ».
upsert_user(uid, email, display_name, ip) — ligne 389

→ Crée l'utilisateur s'il n'existe pas (statut 'pending'), sinon le met à jour.
update_user(uid, display_name) — ligne 407
→ Fonction utilitaire « update_user ».
get_user_by_uid(uid) — ligne 413
→ Fonction utilitaire « get_user_by_uid ».
get_users() — ligne 421
→ Fonction utilitaire « get_users ».
set_user_status(uid, status) — ligne 427
→ Fonction utilitaire « set_user_status ».
set_user_fcm_token(uid, token) — ligne 435
→ Fonction utilitaire « set_user_fcm_token ».
get_setting(key, default) — ligne 442
→ Fonction utilitaire « get_setting ».
set_setting(key, value) — ligne 450
→ Fonction utilitaire « set_setting ».
get_thresholds() — ligne 462
→ Renvoie (seuil_moyen, seuil_eleve), avec valeurs par défaut (25, 55).
set_thresholds(moyen, eleve) — ligne 472
→ Fonction utilitaire « set_thresholds ».
add_audit(actor, action, target, details) — ligne 479
→ Fonction utilitaire « add_audit ».
get_audit(limit) — ligne 487
→ Fonction utilitaire « get_audit ».

Explication ligne par ligne

L.	Code	Explication
1	`"""Couche base de données MoMoGuard — via SQLAlchemy.`	Commentaire / docstring : """Couche base de données MoMoGuard — via SQLAlchemy.
2		Ligne vide (séparation visuelle).
3	`Avantage : on bascule entre SQLite et MySQL (WAMP) en changeant une ligne,`	Instruction Python.
4	`la variable DATABASE_URL ci-dessous. Le reste du code ne change pas.`	Instruction Python.
5		Ligne vide (séparation visuelle).
6	`  - SQLite (par défaut, aucune installation) :`	Instruction Python.
7	`        sqlite:///momoguard.db`	Instruction Python.
8	`  - MySQL via WAMP (décommente et adapte) :`	Instruction Python.
9	`        mysql+pymysql://root:@127.0.0.1:3306/momoguard`	Instruction Python.
10	`    (sous WAMP, l'utilisateur est « root » et le mot de passe est vide par défaut)`	Instruction Python.
11	`"""`	Commentaire / docstring : """
12		Ligne vide (séparation visuelle).
13	`import os`	Import de module ou symbole Python.
14	`import json`	Import de module ou symbole Python.
15	`from datetime import datetime, timezone`	Import de module ou symbole Python.
16		Ligne vide (séparation visuelle).
17	`from sqlalchemy import (create_engine, MetaData, Table, Column, Integer,`	Import de module ou symbole Python.
18	`                        String, Text, select, insert as sa_insert,`	Instruction Python.

L.	Code	Explication
19	`                        update as sa_update, delete as sa_delete,`	Instruction Python.
20	`                        inspect, text)`	Instruction Python.
21		Ligne vide (séparation visuelle).
22	`# === Choix de la base de données ==========================================`	Commentaire / docstring : === Choix de la base de données ==========
23	`_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))`	Affectation de variable ou constante.
24	`_DEFAULT_DB = "sqlite:///" + os.path.join(_ROOT, "momoguard.db").replace("\\", "/")`	Affectation de variable ou constante.
25	`DATABASE_URL = os.environ.get("MOMOGUARD_DB", _DEFAULT_DB)`	Affectation de variable ou constante.

L.	Code	Explication
26	`# Exemple WAMP / MySQL :`	Commentaire / docstring : Exemple WAMP / MySQL :
27	`# DATABASE_URL = "mysql+pymysql://root:@127.0.0.1:3306/momoguard"`	Commentaire / docstring : DATABASE_URL = "mysql+pymysql://root:@127.0.0.1:3306/momoguard"
28	`# ============================================================`	Commentaire / docstring : ==
29		Ligne vide (séparation visuelle).
30	`md = MetaData()`	Affectation de variable ou constante.
31		Ligne vide (séparation visuelle).
32	`reports = Table(`	Affectation de variable ou constante.
33	`    "reports", md,`	Instruction Python.
34	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
35	`    Column("agent_id", String(255), nullable=False),`	Affectation de variable ou constante.
36	`    Column("platform", String(64), nullable=False),`	Affectation de variable ou constante.

L.	Code	Explication
37	`    Column("event_type", String(32), nullable=False),`	Affectation de variable ou constante.
38	`    Column("content", Text, nullable=False),`	Affectation de variable ou constante.
39	`    Column("metadata", Text, nullable=False, default="{}"),`	Affectation de variable ou constante.
40	`    Column("score", Integer, nullable=False, default=0),`	Affectation de variable ou constante.
41	`    Column("level", String(16), nullable=False, default="faible"),`	Affectation de variable ou constante.
42	`    Column("reasons", Text, nullable=False, default="[]"),`	Affectation de variable ou constante.
43	`    Column("created_at", String(40), nullable=False),`	Affectation de variable ou constante.
44	`    Column("uid", String(191)),`	Instruction Python.
45	`)`	Instruction Python.
46		Ligne vide (séparation visuelle).
47	`consents = Table(`	Affectation de variable ou constante.
48	`    "consents", md,`	Instruction Python.
49	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
50	`    Column("participant", String(255), nullable=False),`	Affectation de variable ou constante.
51	`    Column("role", String(64)),`	Instruction Python.
52	`    Column("device", String(255)),`	Instruction Python.
53	`    Column("scope", Text, nullable=False, default="[]"),`	Affectation de variable ou constante.
54	`    Column("agreed_at", String(40), nullable=False),`	Affectation de variable ou constante.

L.	Code	Explication
55	`    Column("revoked_at", String(40)),`	Instruction Python.
56	`)`	Instruction Python.
57		Ligne vide (séparation visuelle).
58	`admins = Table(`	Affectation de variable ou constante.
59	`    "admins", md,`	Instruction Python.
60	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
61	`    Column("username", String(191), nullable=False, unique=True),`	Affectation de variable ou constante.
62	`    Column("password_hash", String(255), nullable=False),`	Affectation de variable ou constante.
63	`    Column("created_at", String(40), nullable=False),`	Affectation de variable ou constante.
64	`    Column("totp_secret", String(64)),`	Instruction Python.
65	`)`	Instruction Python.
66		Ligne vide (séparation visuelle).
67	`agents = Table(`	Affectation de variable ou constante.
68	`    "agents", md,`	Instruction Python.

L.	Code	Explication
69	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
70	`    Column("agent_id", String(191), nullable=False, unique=True),`	Affectation de variable ou constante.
71	`    Column("label", String(255)),`	Instruction Python.
72	`    Column("platform", String(64)),`	Instruction Python.

L.	Code	Explication
73	`    Column("status", String(16), nullable=False, default="active"),`	Affectation de variable ou constante.
74	`    Column("token", String(64)),`	Instruction Python.
75	`    Column("created_at", String(40), nullable=False),`	Affectation de variable ou constante.
76	`    Column("last_seen", String(40)),`	Instruction Python.
77	`)`	Instruction Python.
78		Ligne vide (séparation visuelle).
79	`users = Table(`	Affectation de variable ou constante.
80	`    "users", md,`	Instruction Python.
81	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
82	`    Column("uid", String(191), nullable=False, unique=True),`	Affectation de variable ou constante.
83	`    Column("email", String(255)),`	Instruction Python.
84	`    Column("display_name", String(255)),`	Instruction Python.
85	`    Column("status", String(16), nullable=False, default="pending"),`	Affectation de variable ou constante.
86	`    Column("fcm_token", String(255)),`	Instruction Python.
87	`    Column("ip", String(64)),`	Instruction Python.
88	`    Column("created_at", String(40), nullable=False),`	Affectation de variable ou constante.
89	`    Column("approved_at", String(40)),`	Instruction Python.
90	`    Column("last_seen", String(40)),`	Instruction Python.

L.	Code	Explication
91	`)`	Instruction Python.
92		Ligne vide (séparation visuelle).
93	`settings = Table(`	Affectation de variable ou constante.
94	`    "settings", md,`	Instruction Python.
95	`    Column("skey", String(64), primary_key=True),`	Affectation de variable ou constante.
96	`    Column("svalue", String(255)),`	Instruction Python.
97	`)`	Instruction Python.
98		Ligne vide (séparation visuelle).
99	`audit_log = Table(`	Affectation de variable ou constante.
100	`    "audit_log", md,`	Instruction Python.
101	`    Column("id", Integer, primary_key=True, autoincrement=True),`	Affectation de variable ou constante.
102	`    Column("created_at", String(40), nullable=False),`	Affectation de variable ou constante.
103	`    Column("actor", String(191)),`	Instruction Python.
104	`    Column("action", String(120), nullable=False),`	Affectation de variable ou constante.
105	`    Column("target", String(255)),`	Instruction Python.
106	`    Column("details", Text),`	Instruction Python.
107	`)`	Instruction Python.
108		Ligne vide (séparation visuelle).

L.	Code	Explication
109		Ligne vide (séparation visuelle).
110	`def _make_engine(url):`	Déclaration de la fonction _make_engine().

L.	Code	Explication
111	`    kwargs = {"pool_pre_ping": True}`	Affectation de variable ou constante.
112	`    if url.startswith("sqlite"):`	Condition : branche si la condition est vraie.
113	`        # nécessaire car le serveur tourne en multi-thread`	Commentaire / docstring : nécessaire car le serveur tourne en multi-thread
114	`        kwargs["connect_args"] = {"check_same_thread": False}`	Affectation de variable ou constante.
115	`    return create_engine(url, **kwargs)`	Retourne une valeur au appelant.
116		Ligne vide (séparation visuelle).
117		Ligne vide (séparation visuelle).
118	`engine = _make_engine(DATABASE_URL)`	Affectation de variable ou constante.
119		Ligne vide (séparation visuelle).
120		Ligne vide (séparation visuelle).
121	`def set_database_url(url):`	Déclaration de la fonction set_database_url().
122	`    """Permet de changer de base à chaud (utilisé surtout pour les tests)."""`	Commentaire / docstring : """Permet de changer de base à chaud (utilisé surtout pour les tests)"""
123	`    global engine, DATABASE_URL`	Instruction Python.
124	`    DATABASE_URL = url`	Affectation de variable ou constante.
125	`    engine = _make_engine(url)`	Affectation de variable ou constante.
126		Ligne vide (séparation visuelle).

L.	Code	Explication
127		Ligne vide (séparation visuelle).
128	`def init_db():`	Déclaration de la fonction init_db().
129	`    md.create_all(engine)`	Instruction Python.
130	`    _ensure_column("reports", "uid", "VARCHAR(191)")`	Instruction Python.
131	`    _ensure_column("admins", "totp_secret", "VARCHAR(64)")`	Instruction Python.
132	`    _ensure_column("users", "ip", "VARCHAR(64)")`	Instruction Python.
133	`    _ensure_column("users", "approved_at", "VARCHAR(40)")`	Instruction Python.
134		Ligne vide (séparation visuelle).
135		Ligne vide (séparation visuelle).
136	`def _ensure_column(table_name, column, coltype):`	Déclaration de la fonction _ensure_column().
137	`    """Ajoute une colonne manquante sur une table existante (migration douce)."""`	Commentaire / docstring : """Ajoute une colonne manquante sur une table existante (migration douce).
138	`    try:`	Début bloc try/except (gestion d'erreurs).
139	`        existing = [c["name"] for c in inspect(engine).get_columns(table_name)]`	Affectation de variable ou constante.
140	`    except Exception:`	Capture une exception.
141	`        return`	Instruction Python.
142	`    if column not in existing:`	Condition : branche si la condition est vraie.
143	`        with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
144	`            conn.execute(text(f"ALTER TABLE {table_name} ADD COLUMN {column} {coltype}"))`	Instruction Python.

L.	Code	Explication
145		Ligne vide (séparation visuelle).
146		Ligne vide (séparation visuelle).
147	`def _now():`	Déclaration de la fonction _now().
148	`    return datetime.now(timezone.utc).isoformat()`	Retourne une valeur au appelant.
149		Ligne vide (séparation visuelle).
150		Ligne vide (séparation visuelle).
151	`# --- Rapports ----------------------

L.	Code	Explication
154	`                  score, level, reasons, uid=None):`	Affectation de variable ou constante.
155	`    created_at = _now()`	Affectation de variable ou constante.
156	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
157	`        result = conn.execute(sa_insert(reports).values(`	Affectation de variable ou constante.
158	`            agent_id=agent_id, platform=platform, event_type=event_type,`	Affectation de variable ou constante.
159	`            content=content, metadata=json.dumps(metadata, ensure_ascii=False),`	Affectation de variable ou constante.
160	`            score=score, level=level,`	Affectation de variable ou constante.
161	`            reasons=json.dumps(reasons, ensure_ascii=False),`	Affectation de variable ou constante.
162	`            created_at=created_at, uid=uid,`	Affectation de variable ou constante.

L.	Code	Explication
163	`        ))`	Instruction Python.
164	`        report_id = result.inserted_primary_key[0]`	Affectation de variable ou constante.
165	`    return {`	Retourne une valeur au appelant.
166	`        "id": report_id, "agent_id": agent_id, "platform": platform,`	Instruction Python.
167	`        "event_type": event_type, "content": content, "metadata": metadata,`	Instruction Python.
168	`        "score": score, "level": level, "reasons": reasons,`	Instruction Python.
169	`        "created_at": created_at, "uid": uid,`	Instruction Python.
170	`    }`	Instruction Python.
171		Ligne vide (séparation visuelle).
172		Ligne vide (séparation visuelle).
173	`CONSEILS = [`	Affectation de variable ou constante.
174	`    (("PUK", "SIM"),`	Instruction Python.
175	`     "Un opérateur ne vous demandera jamais votre code PUK. Ignorez…`	Instruction Python.
176	`    (("code", "mot de passe", "OTP"),`	Instruction Python.
177	`     "Ne communiquez jamais votre code de confirmation ni votre mot de passe, même à une person…`	Instruction Python.
178	`    (("agent",),`	Instruction Python.
179	`     "Un vrai agent ne vous demande jamais d'effectuer une opération à distance avec un code.")…`	Instruction Python.
180	`    (("URL", "lien", "domaine", "raccourci", "malveillant"),`	Instruction Python.

L.	Code	Explication
181	`     "Ne cliquez pas sur les liens reçus par SMS : ouvrez vous-même l'application officielle.")…`	Instruction Python.
182	`    (("gain", "loterie", "cadeau"),`	Instruction Python.
183	`     "Une promesse de gain qui réclame des frais à l'avance est une arnaque."),`	Instruction Python.
184	`    (("multiplier",),`	Instruction Python.
185	`     "Aucun service ne multiplie votre argent : « envoyez X, recevez Y » est une arnaque."),`	Instruction Python.
186	`    (("erreur", "rembours"),`	Instruction Python.
187	`     "En cas d'« envoi par erreur », ne renvoyez rien et contactez votre opérateur."),`	Instruction Python.
188	`    (("frais", "déblocage", "debloquer"),`	Instruction Python.
189	`     "Aucun déblocage de compte ne se paie par transfert mobile. C'est une arnaque."),`	Instruction Python.
190	`]`	Instruction Python.
191		Ligne vide (séparation visuelle).
192		Ligne vide (séparation visuelle).
193	`def _conseil(reasons):`	Déclaration de la fonction _conseil().
194	`    """Renvoie un conseil de prévention adapté aux raisons de l'alerte."""`	Commentaire/docstring : """Renvoie un conseil de prévention adapté aux raisons de l'alerte."""
195	`    texte = " ".join(reasons).lower()`	Affectation de variable ou constante.
196	`    for cles, conseil in CONSEILS:`	Boucle de répétition.

L.	Code	Explication
197	`        if any(c.lower() in texte for c in cles):`	Condition : branche si la condition est vraie.
198	`            return conseil`	Retourne une valeur au appelant.

L.	Code	Explication
199	`    return ""`	Retourne une valeur au appelant.
200		Ligne vide (séparation visuelle).
201		Ligne vide (séparation visuelle).
202	`def _report_to_dict(r):`	Déclaration de la fonction _report_to_dict().
203	`    reasons = json.loads(r["reasons"])`	Affectation de variable ou constante.
204	`    return {`	Retourne une valeur au appelant.
205	`        "id": r["id"], "agent_id": r["agent_id"], "platform": r["platform"],`	Instruction Python.
206	`        "event_type": r["event_type"], "content": r["content"],`	Instruction Python.
207	`        "metadata": json.loads(r["metadata"]), "score": r["score"],`	Instruction Python.
208	`        "level": r["level"], "reasons": reasons,`	Instruction Python.
209	`        "conseil": _conseil(reasons),`	Instruction Python.
210	`        "created_at": r["created_at"],`	Instruction Python.
211	`        "uid": r["uid"] if "uid" in r else None,`	Instruction Python.
212	`    }`	Instruction Python.
213		Ligne vide (séparation visuelle).
214		Ligne vide (séparation visuelle).
215	`def get_recent(limit=100):`	Déclaration de la fonction get_recent().
216	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).

L.	Code	Explication
217	`        rows = conn.execute(`	Affectation de variable ou constante.
218	`            select(reports).order_by(reports.c.id.desc()).limit(limit)`	Instruction Python.
219	`        ).mappings().all()`	Instruction Python.
220	`    return [_report_to_dict(r) for r in rows]`	Retourne une valeur au appelant.
221		Ligne vide (séparation visuelle).
222		Ligne vide (séparation visuelle).
223	`def get_reports_for_user(uid, limit=100):`	Déclaration de la fonction get_reports_for_user().
224	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
225	`        rows = conn.execute(`	Affectation de variable ou constante.
226	`            select(reports).where(reports.c.uid == uid)`	Affectation de variable ou constante.
227	`            .order_by(reports.c.id.desc()).limit(limit)`	Instruction Python.
228	`        ).mappings().all()`	Instruction Python.
229	`    return [_report_to_dict(r) for r in rows]`	Retourne une valeur au appelant.
230		Ligne vide (séparation visuelle).
231		Ligne vide (séparation visuelle).
232	`def get_stats():`	Déclaration de la fonction get_stats().
233	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
234	`        rows = conn.execute(`	Affectation de variable ou constante.

L.	Code	Explication
235	`            select(reports.c.level, func.count().label("n")).group_by(reports.c.level)`	Instruction Python.
236	`        ).all()`	Instruction Python.
237	`    stats = {"total": 0, "faible": 0, "moyen": 0, "eleve": 0}`	Affectation de variable ou constante.
238	`    for level, n in rows:`	Boucle de répétition.

L.	Code	Explication
239	`        stats[level] = n`	Affectation de variable ou constante.
240	`        stats["total"] += n`	Affectation de variable ou constante.
241	`    return stats`	Retourne une valeur au appelant.
242		Ligne vide (séparation visuelle).
243		Ligne vide (séparation visuelle).
244	`# --- Consentements ----------------------

L.	Code	Explication
282		Ligne vide (séparation visuelle).
283	`def count_admins():`	Déclaration de la fonction count_admins().
284	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
285	`        return conn.execute(select(func.count()).select_from(admins)).scalar()`	Retourne une valeur au appelant.
286		Ligne vide (séparation visuelle).
287		Ligne vide (séparation visuelle).
288	`def create_admin(username, password_hash):`	Déclaration de la fonction create_admin().

L.	Code	Explication
289	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
290	`        conn.execute(sa_insert(admins).values(`	Instruction Python.
291	`            username=username, password_hash=password_hash,`	Affectation de variable ou constante.
292	`        ))`	Instruction Python.
293		Ligne vide (séparation visuelle).
294		Ligne vide (séparation visuelle).
295	`def get_admin(username):`	Déclaration de la fonction get_admin().
296	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
297	`        row = conn.execute(`	Affectation de variable ou constante.
298	`            select(admins).where(admins.c.username == username)`	Affectation de variable ou constante.
299	`        ).mappings().first()`	Instruction Python.
300	`    return dict(row) if row else None`	Retourne une valeur au appelant.
301		Ligne vide (séparation visuelle).
302		Ligne vide (séparation visuelle).
303	`def update_admin_password(username, password_hash):`	Déclaration de la fonction update_admin_password().
304	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
305	`        conn.execute(`	Instruction Python.
306	`            sa_update(admins).where(admins.c.username == username)`	Affectation de variable ou constante.

L.	Code	Explication
307	`            .values(password_hash=password_hash)`	Affectation de variable ou constante.
308	`        )`	Instruction Python.
309		Ligne vide (séparation visuelle).
310		Ligne vide (séparation visuelle).
311	`def set_admin_totp(username, secret):`	Déclaration de la fonction set_admin_totp().
312	`    """Active (secret) ou désactive (None) la 2FA pour un administrateur."""`	Commentaire / docstring : """Active (secret) ou désactive (None) la 2FA pour un administrateur"""
313	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
314	`        conn.execute(`	Instruction Python.
315	`            sa_update(admins).where(admins.c.username == username)`	Affectation de variable ou constante.
316	`            .values(totp_secret=secret)`	Affectation de variable ou constante.
317	`        )`	Instruction Python.
318		Ligne vide (séparation visuelle).
319		Ligne vide (séparation visuelle).
320	`# --- Agents ----------------------------------------------------------------------`	Commentaire / docstring : --- Agents ----------
321		Ligne vide (séparation visuelle).
322	`def _agent_to_dict(r):`	Déclaration de la fonction _agent_to_dict().
323	`    return {`	Retourne une valeur au appelant.
324	`        "id": r["id"], "agent_id": r["agent_id"], "label": r["label"],`	Instruction Python.

L.	Code	Explication
325	"platform": r["platform"], "status": r["status"], "token": r["token"],	Instruction Python.
326	"created_at": r["created_at"], "last_seen": r["last_seen"],	Instruction Python.
327	}	Instruction Python.
328		Ligne vide (séparation visuelle).
329		Ligne vide (séparation visuelle).
330	def register_agent(agent_id, label, platform, token=None):	Déclaration de la fonction register_agent().
331	existing = get_agent(agent_id)	Affectation de variable ou constante.
332	if existing is None:	Condition : branche si la condition est vraie.
333	with engine.begin() as conn:	Context manager (ressource ouverte/fermée automatiquement).
334	conn.execute(sa_insert(agents).values(	Instruction Python.
335	agent_id=agent_id, label=label, platform=platform,	Affectation de variable ou constante.
336	status="active", token=token, created_at=_now(),	Affectation de variable ou constante.
337	))	Instruction Python.
338	return get_agent(agent_id)	Retourne une valeur au appelant.
339		Ligne vide (séparation visuelle).
340		Ligne vide (séparation visuelle).
341	def get_agent(agent_id):	Déclaration de la fonction get_agent().
342	with engine.connect() as conn:	Context manager (ressource ouverte/fermée automatiquement).

L.	Code	Explication
343	row = conn.execute(	Affectation de variable ou constante.
344	select(agents).where(agents.c.agent_id == agent_id)	Affectation de variable ou constante.
345	).mappings().first()	Instruction Python.
346	return _agent_to_dict(row) if row else None	Retourne une valeur au appelant.
347		Ligne vide (séparation visuelle).
348		Ligne vide (séparation visuelle).
349	def get_agents():	Déclaration de la fonction get_agents().
350	with engine.connect() as conn:	Context manager (ressource ouverte/fermée automatiquement).
351	rows = conn.execute(	Affectation de variable ou constante.
352	select(agents).order_by(agents.c.id.desc())	Instruction Python.
353	).mappings().all()	Instruction Python.
354	return [_agent_to_dict(r) for r in rows]	Retourne une valeur au appelant.
355		Ligne vide (séparation visuelle).
356		Ligne vide (séparation visuelle).
357	def set_agent_status(agent_id, status):	Déclaration de la fonction set_agent_status().
358	with engine.begin() as conn:	Context manager (ressource ouverte/fermée automatiquement).
359	conn.execute(	Instruction Python.
360	sa_update(agents).where(agents.c.agent_id == agent_id).values(status=status)	Affectation de variable ou constante.

L.	Code	Explication
361	)	Instruction Python.
362		Ligne vide (séparation visuelle).
363		Ligne vide (séparation visuelle).
364	def delete_agent(agent_id):	Déclaration de la fonction delete_agent().
365	with engine.begin() as conn:	Context manager (ressource ouverte/fermée automatiquement).
366	conn.execute(sa_delete(agents).where(agents.c.agent_id == agent_id))	Affectation de variable ou constante.
367		Ligne vide (séparation visuelle).

L.	Code	Explication
368		Ligne vide (séparation visuelle).
369	`def touch_agent(agent_id):`	Déclaration de la fonction touch_agent().
370	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
371	`        conn.execute(`	Instruction Python.
372	`            sa_update(agents).where(agents.c.agent_id == agent_id).values(last_seen=_now())`	Affectation de variable ou constante.
373	`        )`	Instruction Python.
374		Ligne vide (séparation visuelle).
375		Ligne vide (séparation visuelle).
376	`# --- Utilisateurs clients (authentifiés via Firebase) ----------`	Commentaire / docstring : --- Utilisateurs clients (authentifiés via Firebase) ----------
377		Ligne vide (séparation visuelle).
378	`def _user_to_dict(r):`	Déclaration de la fonction _user_to_dict().

L.	Code	Explication
379	`    return {`	Retourne une valeur au appelant.
380	`        "id": r["id"], "uid": r["uid"], "email": r["email"],`	Instruction Python.
381	`        "display_name": r["display_name"], "status": r["status"],`	Instruction Python.
382	`        "fcm_token": r["fcm_token"], "created_at": r["created_at"],`	Instruction Python.
383	`        "last_seen": r["last_seen"],`	Instruction Python.
384	`        "ip": r["ip"] if "ip" in r else None,`	Instruction Python.
385	`        "approved_at": r["approved_at"] if "approved_at" in r else None,`	Instruction Python.
386	`    }`	Instruction Python.
387		Ligne vide (séparation visuelle).
388		Ligne vide (séparation visuelle).
389	`def upsert_user(uid, email, display_name, ip=None):`	Déclaration de la fonction upsert_user().
390	`    """Crée l'utilisateur s'il n'existe pas (statut 'pending'), sinon le met à jour."""`	Commentaire / docstring : "Crée l'utilisateur s'il n'existe pas (statut 'pending'), sinon le met à jour."
391	`    existing = get_user_by_uid(uid)`	Affectation de variable ou constante.
392	`    now = _now()`	Affectation de variable ou constante.
393	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
394	`        if existing is None:`	Condition : branche si la condition est vraie.
395	`            conn.execute(sa_insert(users).values(`	Instruction Python.
396	`                uid=uid, email=email, display_name=display_name,`	Affectation de variable ou constante.

L.	Code	Explication
397	`                status="pending", ip=ip, created_at=now, last_seen=now,`	Affectation de variable ou constante.
398	`            ))`	Instruction Python.
399	`        else:`	Branche alternative (sinon).
400	`            vals = {"email": email, "display_name": display_name, "last_seen": now}`	Affectation de variable ou constante.
401	`            if ip:`	Condition : branche si la condition est vraie.
402	`                vals["ip"] = ip`	Affectation de variable ou constante.
403	`            conn.execute(sa_update(users).where(users.c.uid == uid).values(**vals))`	Affectation de variable ou constante.
404	`    return get_user_by_uid(uid)`	Retourne une valeur au appelant.
405		Ligne vide (séparation visuelle).
406		Ligne vide (séparation visuelle).
407	`def update_user(uid, display_name):`	Déclaration de la fonction update_user().
408	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
409	`        conn.execute(sa_update(users).where(users.c.uid == uid)`	Affectation de variable ou constante.
410	`                     .values(display_name=display_name))`	Affectation de variable ou constante.

L.	Code	Explication
411		Ligne vide (séparation visuelle).
412		Ligne vide (séparation visuelle).
413	`def get_user_by_uid(uid):`	Déclaration de la fonction get_user_by_uid().
414	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).

L.	Code	Explication
415	`        row = conn.execute(`	Affectation de variable ou constante.
416	`            select(users).where(users.c.uid == uid)`	Affectation de variable ou constante.
417	`        ).mappings().first()`	Instruction Python.
418	`    return _user_to_dict(row) if row else None`	Retourne une valeur au appelant.
419		Ligne vide (séparation visuelle).
420		Ligne vide (séparation visuelle).
421	`def get_users():`	Déclaration de la fonction get_users().
422	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
423	`        rows = conn.execute(select(users).order_by(users.c.created_at.desc())).mappings().all()`	Affectation de variable ou constante.
424	`    return [_user_to_dict(r) for r in rows]`	Retourne une valeur au appelant.
425		Ligne vide (séparation visuelle).
426		Ligne vide (séparation visuelle).
427	`def set_user_status(uid, status):`	Déclaration de la fonction set_user_status().
428	`    vals = {"status": status}`	Affectation de variable ou constante.
429	`    if status == "approved":`	Condition : branche si la condition est vraie.
430	`        vals["approved_at"] = _now()`	Affectation de variable ou constante.
431	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
432	`        conn.execute(sa_update(users).where(users.c.uid == uid).values(**vals))`	Affectation de variable ou constante.

L.	Code	Explication
433		Ligne vide (séparation visuelle).
434		Ligne vide (séparation visuelle).
435	`def set_user_fcm_token(uid, token):`	Déclaration de la fonction set_user_fcm_token().
436	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
437	`        conn.execute(sa_update(users).where(users.c.uid == uid).values(fcm_token=token))`	Affectation de variable ou constante.
438		Ligne vide (séparation visuelle).
439		Ligne vide (séparation visuelle).
440	`# --- Paramètres (clé/valeur) ----------`	Commentaire / docstring : --- Paramètres (clé/valeur) ----------
441		Ligne vide (séparation visuelle).
442	`def get_setting(key, default=None):`	Déclaration de la fonction get_setting().
443	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
444	`        row = conn.execute(`	Affectation de variable ou constante.
445	`            select(settings.c.svalue).where(settings.c.skey == key)`	Affectation de variable ou constante.
446	`        ).first()`	Instruction Python.
447	`    return row[0] if row else default`	Retourne une valeur au appelant.
448		Ligne vide (séparation visuelle).
449		Ligne vide (séparation visuelle).
450	`def set_setting(key, value):`	Déclaration de la fonction set_setting().

L.	Code	Explication
451	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
452	`        existing = conn.execute(`	Affectation de variable ou constante.

L.	Code	Explication
453	`            select(settings.c.skey).where(settings.c.skey == key)`	Affectation de variable ou constante.
454	`        ).first()`	Instruction Python.
455	`        if existing:`	Condition : branche si la condition est vraie.
456	`            conn.execute(sa_update(settings).where(settings.c.skey == key)`	Affectation de variable ou constante.
457	`                         .values(svalue=str(value)))`	Affectation de variable ou constante.
458	`        else:`	Branche alternative (sinon).
459	`            conn.execute(sa_insert(settings).values(skey=key, svalue=str(value)))`	Affectation de variable ou constante.
460		Ligne vide (séparation visuelle).
461		Ligne vide (séparation visuelle).
462	`def get_thresholds():`	Déclaration de la fonction get_thresholds().
463	`    """Renvoie (seuil_moyen, seuil_eleve), avec valeurs par défaut (25, 55)."""`	Commentaire / docstring : """Renvoie (seuil_moyen, seuil_eleve), avec valeurs par défaut (25, 55).""""
464	`    try:`	Début bloc try/except (gestion d'erreurs).
465	`        moyen = int(get_setting("seuil_moyen", 25))`	Affectation de variable ou constante.
466	`        eleve = int(get_setting("seuil_eleve", 55))`	Affectation de variable ou constante.
467	`    except (TypeError, ValueError):`	Capture une exception.
468	`        moyen, eleve = 25, 55`	Affectation de variable ou constante.

L.	Code	Explication
469	`    return moyen, eleve`	Retourne une valeur au appelant.
470		Ligne vide (séparation visuelle).
471		Ligne vide (séparation visuelle).
472	`def set_thresholds(moyen, eleve):`	Déclaration de la fonction set_thresholds().
473	`    set_setting("seuil_moyen", moyen)`	Instruction Python.
474	`    set_setting("seuil_eleve", eleve)`	Instruction Python.
475		Ligne vide (séparation visuelle).
476		Ligne vide (séparation visuelle).
477	`# --- Journal d'audit (non-répudiation) ----------`	Commentaire / docstring : --- Journal d'audit (non-répudiation) ----------
478		Ligne vide (séparation visuelle).
479	`def add_audit(actor, action, target=None, details=None):`	Déclaration de la fonction add_audit().
480	`    with engine.begin() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
481	`        conn.execute(sa_insert(audit_log).values(`	Instruction Python.
482	`            created_at=_now(), actor=actor or "?", action=action,`	Affectation de variable ou constante.
483	`            target=target, details=details,`	Affectation de variable ou constante.
484	`        ))`	Instruction Python.
485		Ligne vide (séparation visuelle).
486		Ligne vide (séparation visuelle).

L.	Code	Explication
487	`def get_audit(limit=200):`	Déclaration de la fonction get_audit().
488	`    with engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
489	`        rows = conn.execute(`	Affectation de variable ou constante.
490	`            select(audit_log).order_by(audit_log.c.id.desc()).limit(limit)`	Instruction Python.
491	`        ).mappings().all()`	Instruction Python.
492	`    return [dict(r) for r in rows]`	Retourne une valeur au appelant.

Fichier : server/scoring.py

Moteur de détection par règles explicables.

Fonctions déclarées

_sans_accents(texte) — ligne 77

→ Minuscule + suppression des accents (gagné -> gagne).

_extract_domain(url) — ligne 83

→ Fonction utilitaire « _extract_domain ».

_domaine_nettoye(domain) — ligne 89

→ Ramène un domaine à sa forme « lisible » pour détecter les sosies.

score_event(event_type, content, metadata, thresholds) — ligne 95

→ Analyse un évènement et renvoie (score, niveau, raisons).
thresholds = (seuil_moyen, seuil_eleve) : bornes configurables des niveaux.

Explication ligne par ligne

L.	Code	Explication
1	`"""Moteur de scoring basé sur des règles (explicable) — version affinée.`	Commentaire / docstring : """Moteur de scoring basé sur des règles (explicable) — version affinée.
2		Ligne vide (séparation visuelle).
3	`Principe : chaque signal suspect ajoute des points. La somme donne un score`	Instruction Python.
4	`(0-100) et un niveau (faible / moyen / eleve). Chaque règle déclenchée est`	Instruction Python.
5	`enregistrée dans `reasons` pour pouvoir EXPLIQUER la décision devant le jury.`	Instruction Python.
6		Ligne vide (séparation visuelle).
7	`Améliorations de cette version :`	Instruction Python.
8	`  - détection des domaines sosies écrits en homoglyphes (orange.cd -> 0range.cd, v0dacom) ;`	Instruction Python.
9	`  - correspondance des mots-clés insensible aux accents (gagné = gagne) ;`	Affectation de variable ou constante.
10	`  - bonus quand urgence et lien apparaissent ensemble (schéma classique).`	Instruction Python.
11	`"""`	Commentaire / docstring : """
12		Ligne vide (séparation visuelle).
13	`import re`	Import de module ou symbole Python.
14	`import unicodedata`	Import de module ou symbole Python.
15		Ligne vide (séparation visuelle).
16	`# --- Listes de référence (contexte Mobile Money RDC) ---`	Commentaire / docstring : --- Listes de référence (contexte Mobile Money RDC) ----------
17		Ligne vide (séparation visuelle).
18	`OPERATEURS = ["orange", "airtel", "vodacom", "mpesa", "afrimoney"]`	Affectation de variable ou constante.

L.	Code	Explication
19		Ligne vide (séparation visuelle).
20	`# Domaines officiels (approximatifs, à compléter selon tes sources réelles)`	Commentaire / docstring : Domaines officiels (approximatifs, à compléter selon tes sources réelles)
21	`DOMAINES_OFFICIELS = ["orange.cd", "airtel.cd", "vodacom.cd", "rawbank.cd"]`	Affectation de variable ou constante.
22		Ligne vide (séparation visuelle).
23	`TLD_SUSPECTS = [".xyz", ".top", ".tk", ".ml", ".ga", ".cf", ".gq", ".click"]`	Affectation de variable ou constante.
24		Ligne vide (séparation visuelle).
25	`RACCOURCISSEURS = ["bit.ly", "tinyurl.com", "t.co", "goo.gl", "ow.ly", "is.gd", "cutt.ly"]`	Affectation de variable ou constante.
26		Ligne vide (séparation visuelle).
27	`# Mots-clés écrits SANS accent : la comparaison se fait sur du texte normalisé.`	Commentaire / docstring : Mots-clés écrits SANS accent : la comparaison se fait sur du texte normalisé.
28	`MOTS_CLES_FRAUDE = [`	Affectation de variable ou constante.
29	`    "code pin", "code secret", "code de confirmation", "urgent", "gagner",`	Instruction Python.
30	`    "gagnant", "felicitations", "cliquez", "bloque", "reactiver", "confirmez",`	Instruction Python.

L.	Code	Explication
31	`    "verifiez", "promotion", "tombola", "loterie", "recompense", "cadeau",`	Instruction Python.
32	`    "votre compte sera", "activer maintenant", "lien ci-dessous",`	Instruction Python.
33	`]`	Instruction Python.
34		Ligne vide (séparation visuelle).
35	`MOTS_URGENCE = ["urgent", "immediatement", "maintenant", "aujourd'hui", "vite", "expire"]`	Affectation de variable ou constante.
36		Ligne vide (séparation visuelle).

L.	Code	Explication
37	`# Nouveaux types d'attaque (Phase 5)`	Commentaire / docstring : Nouveaux types d'attaque (Phase 5)
38	`MOTS_SIM_SWAP = [`	Affectation de variable ou constante.
39	`    "carte sim", "nouvelle sim", "nouvelle carte sim", "remplacer ma sim",`	Instruction Python.
40	`    "remplacement de sim", "remplacement sim", "code puk",`	Instruction Python.
41	`    "changer de sim", "swap sim", "sim bloquee", "reinitialiser la sim",`	Instruction Python.
42	`]`	Instruction Python.
43	`VERBES_PARTAGE = [`	Affectation de variable ou constante.
44	`    "partage", "partagez", "envoie", "envoyez", "donne", "donnez",`	Instruction Python.
45	`    "communique", "communiquez", "transfere", "transferez", "dicte", "dictez",`	Instruction Python.
46	`]`	Instruction Python.
47	`MOTS_CODE = ["code", "otp", "code de confirmation", "code de verification", "code secret"]`	Affectation de variable ou constante.
48	`MOTS_ACTION_AGENT = ["composez", "envoyez", "confirmez", "validez", "donnez", "communiquez", "transfer…`	Affectation de variable ou constante.
49	`MOTS_ERREUR = ["par erreur", "envoye par erreur", "erreur", "transfere par erreur", "envoyé par erreur"…`	Affectation de variable ou constante.
50	`MOTS_REMBOURSEMENT = ["rembours", "renvoyez", "renvoie", "retournez", "remboursement", "argent", "re…`	Affectation de variable ou constante.
51		Ligne vide (séparation visuelle).
52	`# Identifiants sensibles (un code OU un mot de passe ne se partagent jamais)`	Commentaire / docstring : Identifiants sensibles (un code OU un mot de passe ne se partagent jamais)
53	`MOTS_IDENTIFIANT = MOTS_CODE + ["mot de passe", "code pin", "otp"]`	Affectation de variable ou constante.
54	`# Formulations PROTECTRICES (le message met en garde, il ne demande pas)`	Commentaire / docstring : Formulations PROTECTRICES (le message met en garde, il ne demande pas)

L.	Code	Explication
55	`MOTS_PROTECTEUR = ["jamais", "a personne", "ne communiquez", "ne donnez",`	Affectation de variable ou constante.
56	`                   "ne partagez", "ne le partagez", "ne donnez", "ne divulguez"…`	Instruction Python.
57	`# Appât au gain (loterie / cadeau) et amorces de sollicitation`	Commentaire / docstring : Appât au gain (loterie / cadeau) et amorces de sollicitation
58	`MOTS_GAIN = ["gagne", "gagnant", "gagner", "tombola", "loterie", "felicitations",`	Affectation de variable ou constante.
59	`              "selectionne", "tirage", "recompense", "cadeau"]`	Instruction Python.
60	`MOTS_APPAT = ["frais", "payer", "envoyez", "envoie", "recevez", "debloquer", "bonus"]`	Affectation de variable ou constante.
61		Ligne vide (séparation visuelle).
62	`EXTENSIONS_DANGEREUSES = [".exe", ".scr", ".bat", ".cmd", ".vbs", ".jar", ".apk", ".msi"]`	Affectation de variable ou constante.
63		Ligne vide (séparation visuelle).
64	`# Substitutions d'homoglyphes les plus courantes`	Commentaire / docstring : Substitutions d'homoglyphes les plus courantes
65	`HOMOGLYPHES = str.maketrans({`	Affectation de variable ou constante.
66	`    "0": "o", "1": "i", "3": "e", "4": "a", "5": "s",`	Instruction Python.
67	`    "7": "t", "8": "b", "$": "s", "@": "a",`	Instruction Python.
68	`})`	Instruction Python.
69		Ligne vide (séparation visuelle).
70	`URL_REGEX = re.compile(r"(https?://[^\s]+|www\.[^\s]+)", re.IGNORECASE)`	Affectation de variable ou constante.
71	`MONTANT_REGEX = re.compile(r"\d[\d.\s]*\s*(\$|fc|usd|cdf)", re.IGNORECASE)`	Affectation de variable ou constante.
72	`IP_HOST_REGEX = re.compile(r"https?://\d{1,3}(\.\d{1,3}){3}", re.IGNORECASE)`	Affectation de variable ou constante.

L.	Code	Explication
73	`USSD_REGEX = re.compile(r"\*\d{2,4}(\*[\d#]+)*#")`	Affectation de variable ou constante.
74	`NUMERO_REGEX = re.compile(r"(\+?243|0)\d{8,9}")`	Affectation de variable ou constante.
75		Ligne vide (séparation visuelle).
76		Ligne vide (séparation visuelle).
77	`def _sans_accents(texte):`	Déclaration de la fonction _sans_accents().
78	`"""Minuscule + suppression des accents (gagné -> gagne)."""`	Commentaire / docstring : """Minuscule + suppression des accents (gagné -> gagne)"""
79	`t = unicodedata.normalize("NFD", texte.lower())`	Affectation de variable ou constante.
80	`return "".join(c for c in t if unicodedata.category(c) != "Mn")`	Retourne une valeur au appelant.
81		Ligne vide (séparation visuelle).
82		Ligne vide (séparation visuelle).
83	`def _extract_domain(url):`	Déclaration de la fonction _extract_domain().
84	`url = re.sub(r"^https?://", "", url, flags=re.IGNORECASE)`	Affectation de variable ou constante.
85	`url = re.sub(r"^www\.", "", url, flags=re.IGNORECASE)`	Affectation de variable ou constante.
86	`return url.split("/")[0].lower()`	Retourne une valeur au appelant.
87		Ligne vide (séparation visuelle).
88		Ligne vide (séparation visuelle).
89	`def _domaine_nettoye(domain):`	Déclaration de la fonction _domaine_nettoye().
90	`"""Ramène un domaine à sa forme « lisible » pour détecter les sosies."""`	Commentaire / docstring : """Ramène un domaine à sa forme « lisible » pour détecter les sosies."""

L.	Code	Explication
91	`deleet = domain.translate(HOMOGLYPHES)`	Affectation de variable ou constante.
92	`return re.sub(r"[^a-z]", "", deleet)`	Retourne une valeur au appelant.
93		Ligne vide (séparation visuelle).
94		Ligne vide (séparation visuelle).
95	`def score_event(event_type, content, metadata=None, thresholds=(25, 55)):`	Déclaration de la fonction score_event().
96	`"""Analyse un évènement et renvoie (score, niveau, raisons).`	Commentaire / docstring : """Analyse un évènement et renvoie (score, niveau, raisons).
97		Ligne vide (séparation visuelle).
98	`thresholds = (seuil_moyen, seuil_eleve) : bornes configurables.`	Affectation de variable ou constante.
99	`"""`	Commentaire / docstring : """
100	`metadata = metadata or {}`	Affectation de variable ou constante.
101	`content = content or ""`	Affectation de variable ou constante.
102	`norm = _sans_accents(content)`	Affectation de variable ou constante.
103	`score = 0`	Affectation de variable ou constante.
104	`reasons = []`	Affectation de variable ou constante.
105		Ligne vide (séparation visuelle).
106	`# 1) Analyse des liens / URLs ----------`	Commentaire / docstring : 1) Analyse des liens / URLs ----------
107	`urls = URL_REGEX.findall(content)`	Affectation de variable ou constante.
108	`urls = [u[0] if isinstance(u, tuple) else u for u in urls]`	Affectation de variable ou constante.

L.	Code	Explication
109	`a_un_lien = bool(urls)`	Affectation de variable ou constante.
110	`for url in urls:`	Boucle de répétition.
111	`domain = _extract_domain(url)`	Affectation de variable ou constante.
112	`clean = _domaine_nettoye(domain)`	Affectation de variable ou constante.
113		Ligne vide (séparation visuelle).
114	`if IP_HOST_REGEX.search(url):`	Condition : branche si la condition est vraie.
115	`score += 30`	Affectation de variable ou constante.

L.	Code	Explication
116	`            reasons.append("Lien pointant vers une adresse IP brute (très suspect)")`	Instruction Python.
117		Ligne vide (séparation visuelle).
118	`        if any(domain == d or domain.endswith("." + d) for d in RACCOURCISSEURS):`	Condition : branche si la condition est vraie.
119	`            score += 20`	Affectation de variable ou constante.
120	`            reasons.append(f"Lien raccourci masquant la vraie destination ({domain})")`	Instruction Python.
121		Ligne vide (séparation visuelle).
122	`        if any(domain.endswith(tld) for tld in TLD_SUSPECTS):`	Condition : branche si la condition est vraie.
123	`            score += 15`	Affectation de variable ou constante.
124	`            reasons.append(f"Extension de domaine peu fiable ({domain})")`	Instruction Python.
125		Ligne vide (séparation visuelle).
126	`        # Domaine sosie : imite un opérateur (même via homoglyphes) sans être officiel`	Commentaire / docstring : Domaine sosie : imite un opérateur (même via homoglyphes) sans être offi

L.	Code	Explication
127	`        if any(op in clean for op in OPERATEURS) and not any(`	Condition : branche si la condition est vraie.
128	`            domain == d or domain.endswith("." + d) for d in DOMAINES_OFFICIELS`	Affectation de variable ou constante.
129	`        ):`	Instruction Python.
130	`            score += 35`	Affectation de variable ou constante.
131	`            note = f"Domaine imitant un opérateur sans être officiel ({domain})"`	Affectation de variable ou constante.
132	`            if clean != re.sub(r"[^a-z]", "", domain):`	Condition : branche si la condition est vraie.
133	`                note += f" → lu « {clean} »"`	Affectation de variable ou constante.
134	`            reasons.append(note + ")")`	Instruction Python.
135		Ligne vide (séparation visuelle).
136	`        if url.lower().startswith("http://"):`	Condition : branche si la condition est vraie.
137	`            score += 5`	Affectation de variable ou constante.
138	`            reasons.append("Connexion non sécurisée (http au lieu de https)")`	Instruction Python.
139		Ligne vide (séparation visuelle).
140	`    # 2) Mots-clés d'ingénierie sociale (insensible aux accents) ----------`	Commentaire / docstring : 2) Mots-clés d'ingénierie sociale (insensible aux accents) ----------
141	`    cles_trouvees = [k for k in MOTS_CLES_FRAUDE if k in norm]`	Affectation de variable ou constante.
142	`    if cles_trouvees:`	Condition : branche si la condition est vraie.
143	`        gain = min(len(cles_trouvees) * 8, 32)`	Affectation de variable ou constante.
144	`        score += gain`	Affectation de variable ou constante.

L.	Code	Explication
145	`        reasons.append("Vocabulaire d'arnaque détecté : " + ", ".join(cl...`	Instruction Python.
146		Ligne vide (séparation visuelle).
147	`    # 3) Combo urgence + lien (schéma classique de phishing) ----------`	Commentaire / docstring : 3) Combo urgence + lien (schéma classique de phishing) ----------
148	`    if a_un_lien and any(u in norm for u in MOTS_URGENCE):`	Condition : branche si la condition est vraie.
149	`        score += 12`	Affectation de variable ou constante.
150	`        reasons.append("Sentiment d'urgence combiné à un lien (schéma typique d'arnaque)")`	Instruction Python.
151		Ligne vide (séparation visuelle).
152	`    # 4) Expéditeur usurpé ----------`	Commentaire / docstring : 4) Expéditeur usurpé ----------
153	`    sender = str(metadata.get("sender", "")).strip()`	Affectation de variable ou constante.
154	`    if sender and NUMERO_REGEX.fullmatch(sender) and any(op in norm for op in OPERATEURS):`	Condition : branche si la condition est vraie.
155	`        score += 15`	Affectation de variable ou constante.
156	`        reasons.append(f"Message au nom d'un opérateur envoyé depuis un numéro personnel ({send...`	Instruction Python.
157		Ligne vide (séparation visuelle).
158	`    # 5) Code USSD à risque ----------`	Commentaire / docstring : 5) Code USSD à risque ----------

L.	Code	Explication
159	`    if USSD_REGEX.search(content):`	Condition : branche si la condition est vraie.
160	`        score += 25`	Affectation de variable ou constante.
161	`        reasons.append("Présence d'un code USSD (possible transfert non envoi dicté)")`	Instruction Python.
162		Ligne vide (séparation visuelle).

L.	Code	Explication
163	`    # 6) Fichier dangereux ----------`	Commentaire / docstring : 6) Fichier dangereux ----------
164	`    if event_type == "file" and any(norm.endswith(ext) for ext in EXTENSIONS_DANGEREUSES):`	Condition : branche si la condition est vraie.
165	`        score += 30`	Affectation de variable ou constante.
166	`        reasons.append("Fichier exécutable potentiellement malveillant")`	Instruction Python.
167		Ligne vide (séparation visuelle).
168	`    # 7) Presse-papier contenant un numéro de paiement ----------`	Commentaire / docstring : 7) Presse-papier contenant un numéro de paiement ----------
169	`    if event_type == "clipboard" and NUMERO_REGEX.search(content):`	Condition : branche si la condition est vraie.
170	`        score += 10`	Affectation de variable ou constante.
171	`        reasons.append("Numéro de compte/téléphone copié récemment (vérifier qu'il n'a pas été modifié")`	Instruction Python.
172		Ligne vide (séparation visuelle).
173	`    # 8) SIM swap (remplacement de SIM / PUK) ----------`	Commentaire / docstring : 8) SIM swap (remplacement de SIM / PUK) ----------
174	`    if any(k in norm for k in MOTS_SIM_SWAP):`	Condition : branche si la condition est vraie.
175	`        score += 25`	Affectation de variable ou constante.
176	`        reasons.append("Référence à un remplacement de SIM/code PUK (possible SIM swap)")`	Instruction Python.
177		Ligne vide (séparation visuelle).
178	`    # 9) Partage ou demande d'un identifiant (code OTP ou mot de passe) ----------`	Commentaire / docstring : 9) Partage ou demande d'un identifiant (code OTP ou mot de passe) ----------
179	`    if (any(k in norm for k in MOTS_IDENTIFIANT) and any(v in norm for v in VERBES_PARTAGE)`	Condition : branche si la condition est vraie.
180	`            and not any(p in norm for p in MOTS_PROTECTEURS)):`	Instruction Python.

L.	Code	Explication
181	`        score += 30`	Affectation de variable ou constante.
182	`        reasons.append("Demande de partage d'un code/mot de passe (on ne partagent jamais)")`	Instruction Python.
183		Ligne vide (séparation visuelle).
184	`    # 10) Faux agent Mobile Money ----------`	Commentaire / docstring : 10) Faux agent Mobile Money ----------
185	`    if ("agent" in norm and any(op in norm for op in OPERATEURS)`	Condition : branche si la condition est vraie.
186	`            and any(a in norm for a in MOTS_ACTION_AGENT)):`	Instruction Python.
187	`        score += 25`	Affectation de variable ou constante.
188	`        reasons.append("Message se présentant comme un agent opérateur avec une consigne d'acti…`	Instruction Python.
189		Ligne vide (séparation visuelle).
190	`    # 11) Arnaque « envoi par erreur / remboursement » ----------`	Commentaire / docstring : 11) Arnaque « envoi par erreur / remboursement » ----------
191	`    if any(e in norm for e in MOTS_ERREUR) and any(r in norm for r in MOTS_REMBOURSEMENT):`	Condition : branche si la condition est vraie.
192	`        score += 25`	Affectation de variable ou constante.
193	`        reasons.append("Schéma « envoi par erreur, remboursement » (arnaque au remboursement)"…`	Instruction Python.
194		Ligne vide (séparation visuelle).
195	`    # 12) Appât au gain (loterie / cadeau) avec sollicitation ----------`	Commentaire / docstring : 12) Appât au gain (loterie / cadeau) avec sollicitation ----------
196	`    if any(g in norm for g in MOTS_GAIN) and any(a in norm for a in MOTS_APPAT):`	Condition : branche si la condition est vraie.
197	`        score += 25`	Affectation de variable ou constante.
198	`        reasons.append("Appât au gain (loterie/cadeau) accompagné d'une sollicitation")`	Instruction Python.

L.	Code	Explication
199		Ligne vide (séparation visuelle).
200	`    # 13) Promesse de multiplier l'argent ----------`	Commentaire / docstring : 13) Promesse de multiplier l'argent ----------

L.	Code	Explication
201	`    if ("recevez" in norm and ("envoyez" in norm or "envoi" in norm)`	Condition : branche si la condition est vraie.
202	`            and MONTANT_REGEX.search(norm)):`	Instruction Python.
203	`        score += 22`	Affectation de variable ou constante.
204	`        reasons.append("Promesse de multiplier l'argent (« envoyez X, recevez Y »)")`	Instruction Python.
205		Ligne vide (séparation visuelle).
206	`    # 14) Frais à payer pour débloquer un compte ----------`	Commentaire /-docstring : 14) Frais à payer pour débloquer un compte ----------
207	`    if "frais" in norm and ("payer" in norm or "debloquer" in norm):`	Condition : branche si la condition est vraie.
208	`        score += 22`	Affectation de variable ou constante.
209	`        reasons.append("Demande de frais pour débloquer un compte (typique aux frais)")`	Instruction Python.
210		Ligne vide (séparation visuelle).
211	`    # --- Bornage et niveau ----------`	Commentaire /-docstring : --- Bornage et niveau ----------
212	`    seuil_moyen, seuil_eleve = thresholds`	Affectation de variable ou constante.
213	`    score = max(0, min(score, 100))`	Affectation de variable ou constante.
214	`    if score >= seuil_eleve:`	Condition : branche si la condition est vraie.
215	`        level = "eleve"`	Affectation de variable ou constante.
216	`    elif score >= seuil_moyen:`	Condition : branche si la condition est vraie.

L.	Code	Explication
217	`        level = "moyen"`	Affectation de variable ou constante.
218	`    else:`	Branche alternative (sinon).
219	`        level = "faible"`	Affectation de variable ou constante.
220		Ligne vide (séparation visuelle).
221	`    if not reasons:`	Condition : branche si la condition est vraie.
222	`        reasons.append("Aucun signal suspect détecté")`	Instruction Python.
223		Ligne vide (séparation visuelle).
224	`    return score, level, reasons`	Retourne une valeur au appelant.

Fichier : server/threat_intel.py

Vérification URL via Google Safe Browsing.

Fonctions déclarées

is_ready() — ligne 21

→ Vrai si une clé API est configurée.

extract_urls(text) — ligne 26

→ Extrait les URLs d'un texte (préfixe http:// pour les www.).

check_urls(urls) — ligne 37

→ Renvoie 'malicious' si au moins une URL est signalée, 'clean' si toutes sont saines, 'unknown' sans clé ou en cas d'erreur.

Explication ligne par ligne

L.	Code	Explication
1	`"""Consommation d'une API externe de réputation d'URL (threat intelligence)`	Commentaire / docstring : """Consommation d'une API externe de réputation d'URL (threat intelligenc
2		Ligne vide (séparation visuelle).
3	`Interroge Google Safe Browsing (v4) pour savoir si une URL est connue comme`	Instruction Python.
4	`malveillante (phishing, malware, logiciel indésirable). La clé API est lue`	Instruction Python.
5	`via la variable d'environnement SAFEBROWSING_API_KEY.`	Instruction Python.
6		Ligne vide (séparation visuelle).
7	`Dégradation gracieuse : sans clé ou en cas d'erreur réseau, le module renvoie`	Instruction Python.
8	`« unknown » et n'affecte pas le scoring — la plateforme continue de fonctionner`	Instruction Python.
9	`hors ligne.`	Instruction Python.
10	`"""`	Commentaire / docstring : """
11	`import os`	Import de module ou symbole Python.
12	`import re`	Import de module ou symbole Python.
13		Ligne vide (séparation visuelle).
14	`import requests`	Import de module ou symbole Python.
15		Ligne vide (séparation visuelle).
16	`API_URL = "https://safebrowsing.googleapis.com/v4/threatMatches:find"`	Affectation de variable ou constante.
17	`URL_REGEX = re.compile(r"(https?://[^\s]+|www\.[^\s]+)", re.IGNORECASE)`	Affectation de variable ou constante.
18	`TIMEOUT = 4`	Affectation de variable ou constante.

L.	Code	Explication
19		Ligne vide (séparation visuelle).
20		Ligne vide (séparation visuelle).
21	`def is_ready():`	Déclaration de la fonction is_ready().
22	`    """Vrai si une clé API est configurée."""`	Commentaire / docstring : """Vrai si une clé API est configurée."""
23	`    return bool(os.environ.get("SAFEBROWSING_API_KEY"))`	Retourne une valeur au appelant.
24		Ligne vide (séparation visuelle).
25		Ligne vide (séparation visuelle).
26	`def extract_urls(text):`	Déclaration de la fonction extract_urls().
27	`    """Extrait les URLs d'un texte (préfixe http:// pour les www.)."""`	Commentaire / docstring : """Extrait les URLs d'un texte (préfixe http:// pour les www.)."""
28	`    urls = []`	Affectation de variable ou constante.
29	`    for match in URL_REGEX.findall(text or ""):`	Boucle de répétition.
30	`        url = match[0] if isinstance(match, tuple) else match`	Affectation de variable ou constante.
31	`        if url.lower().startswith("www."):`	Condition : branche si la condition est vraie.
32	`            url = "http://" + url`	Affectation de variable ou constante.

L.	Code	Explication
33	`        urls.append(url.rstrip(".,;)"))`	Instruction Python.
34	`    return urls`	Retourne une valeur au appelant.
35		Ligne vide (séparation visuelle).
36		Ligne vide (séparation visuelle).

L.	Code	Explication
37	`def check_urls(urls):`	Déclaration de la fonction check_urls().
38	`    """Renvoie 'malicious' si au moins une URL est signalée, 'clean' si toutes`	Commentaire / docstring: """Renvoie 'malicious' si au moins une URL est signalée, 'clean' si toutes
39	`    sont saines, 'unknown' sans clé ou en cas d'erreur."""`	Instruction Python.
40	`    key = os.environ.get("SAFEBROWSING_API_KEY")`	Affectation de variable ou constante.
41	`    if not key or not urls:`	Condition : branche si la condition est vraie.
42	`        return "unknown"`	Retourne une valeur au appelant.
43	`    body = {`	Affectation de variable ou constante.
44	`        "client": {"clientId": "momoguard", "clientVersion": "1.0"},`	Instruction Python.
45	`        "threatInfo": {`	Instruction Python.
46	`            "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING", "UNWANTED_SOFTWARE"],`	Instruction Python.
47	`            "platformTypes": ["ANY_PLATFORM"],`	Instruction Python.
48	`            "threatEntryTypes": ["URL"],`	Instruction Python.
49	`            "threatEntries": [{"url": u} for u in urls],`	Instruction Python.
50	`        },`	Instruction Python.
51	`    }`	Instruction Python.
52	`    try:`	Début bloc try/except (gestion d'erreurs).
53	`        r = requests.post(API_URL, params={"key": key}, json=body, timeout=TIMEOUT)`	Affectation de variable ou constante.
54	`        if r.status_code != 200:`	Condition : branche si la condition est vraie.

L.	Code	Explication
55	`            return "unknown"`	Retourne une valeur au appelant.
56	`        return "malicious" if (r.json() or {}).get("matches") else "clean"`	Retourne une valeur au appelant.
57	`    except Exception:`	Capture une exception.
58	`        return "unknown"`	Retourne une valeur au appelant.

Fichier : server/firebase_auth.py

Auth mobile Firebase + notifications push.

Fonctions déclarées

init_firebase(cred_path) — ligne 22

→ Initialise le SDK si la clé existe. Renvoie True si prêt.

is_ready() — ligne 41

→ Fonction utilitaire « is_ready ».

verify_id_token(id_token) — ligne 45

→ Vérifie un ID token Firebase et renvoie le dict décodé (uid, email, name).
Lève une exception si le jeton est invalide ou expiré.

send_push(fcm_token, title, body) — ligne 54

→ Envoie une notification push à un appareil via Firebase Cloud Messaging.
Renvoie True si l'envoi a réussi. Ne lève jamais : en cas d'échec, log + False.

update_user_password(uid, new_password) — ligne 74

→ Met à jour le mot de passe d'un utilisateur Firebase (côté admin).
Renvoie True si réussi. Lève une exception si Firebase n'est pas prêt ou en

Explication ligne par ligne

L.	Code	Explication
1	`"""Vérification des jetons Firebase côté serveur (SDK firebase-admin)."""`	Commentaire / docstring : """Vérification des jetons Firebase côté serveur (SDK firebase-admin).
2		Ligne vide (séparation visuelle).
3	`L'app Android authentifie l'utilisateur via Firebase, récupère un « ID token »,`	Instruction Python.
4	`et l'envoie au backend. Ce module vérifie ce jeton et en extrait l'identité.`	Instruction Python.
5		Ligne vide (séparation visuelle).
6	`Le serveur a besoin d'une clé de compte de service Firebase :`	Instruction Python.
7	`  Console Firebase → Paramètres du projet → Comptes de service →`	Instruction Python.
8	`  « Générer une nouvelle clé privée » → place le fichier ici sous le nom`	Instruction Python.
9	`  serviceAccountKey.json (NE PAS le mettre sur GitHub).`	Instruction Python.
10		Ligne vide (séparation visuelle).
11	`Si la clé est absente, l'authentification Firebase est simplement désactivée`	Instruction Python.
12	`et le reste de la plateforme continue de fonctionner normalement.`	Instruction Python.
13	`"""`	Commentaire / docstring : """
14		Ligne vide (séparation visuelle).
15	`import os`	Import de module ou symbole Python.
16		Ligne vide (séparation visuelle).
17	`ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))`	Affectation de variable ou constante.
18	`CRED_PATH = os.path.join(ROOT, "serviceAccountKey.json")`	Affectation de variable ou constante.

L.	Code	Explication
19	`_initialized = False`	Affectation de variable ou constante.
20		Ligne vide (séparation visuelle).
21		Ligne vide (séparation visuelle).
22	`def init_firebase(cred_path=CRED_PATH):`	Déclaration de la fonction init_firebase().
23	`    """Initialise le SDK si la clé existe. Renvoie True si prêt."""`	Commentaire / docstring : """Initialise le SDK si la clé existe. Renvoie True si prêt."""
24	`    global _initialized`	Instruction Python.
25	`    if _initialized:`	Condition : branche si la condition est vraie.
26	`        return True`	Retourne une valeur au appelant.
27	`    if not os.path.exists(cred_path):`	Condition : branche si la condition est vraie.
28	`        return False`	Retourne une valeur au appelant.

L.	Code	Explication
29	`    try:`	Début bloc try/except (gestion d'erreurs).
30	`        import firebase_admin`	Import de module ou symbole Python.
31	`        from firebase_admin import credentials`	Import de module ou symbole Python.
32	`        cred = credentials.Certificate(cred_path)`	Affectation de variable ou constante.
33	`        firebase_admin.initialize_app(cred)`	Instruction Python.
34	`        _initialized = True`	Affectation de variable ou constante.
35	`        return True`	Retourne une valeur au appelant.
36	`    except Exception as e:`	Capture une exception.

L.	Code	Explication
37	`        print(f"[Firebase] initialisation impossible : {e}")`	Instruction Python.
38	`        return False`	Retourne une valeur au appelant.
39		Ligne vide (séparation visuelle).
40		Ligne vide (séparation visuelle).
41	`def is_ready():`	Déclaration de la fonction is_ready().
42	`    return _initialized`	Retourne une valeur au appelant.
43		Ligne vide (séparation visuelle).
44		Ligne vide (séparation visuelle).
45	`def verify_id_token(id_token):`	Déclaration de la fonction verify_id_token().
46	`    """Vérifie un ID token Firebase et renvoie le dict décodé (uid, email, name).`	Commentaire / docstring : """Vérifie un ID token Firebase et renvoie le dict décodé (uid, email, name).
47		Ligne vide (séparation visuelle).
48	`    Lève une exception si le jeton est invalide ou expiré.`	Instruction Python.
49	`    """`	Commentaire / docstring : """
50	`    from firebase_admin import auth`	Import de module ou symbole Python.
51	`    return auth.verify_id_token(id_token)`	Retourne une valeur au appelant.
52		Ligne vide (séparation visuelle).
53		Ligne vide (séparation visuelle).
54	`def send_push(fcm_token, title, body):`	Déclaration de la fonction send_push().

L.	Code	Explication
55	`    """Envoie une notification push à un appareil via Firebase Cloud Messaging.`	Commentaire / docstring : """Envoie une notification push à un appareil via Firebase Cloud Messaging.
56		Ligne vide (séparation visuelle).
57	`    Renvoie True si l'envoi a réussi. Ne lève jamais : en cas d'échec, log + False.`	Instruction Python.
58	`    """`	Commentaire / docstring : """
59	`    if not _initialized or not fcm_token:`	Condition : branche si la condition est vraie.
60	`        return False`	Retourne une valeur au appelant.
61	`    try:`	Début bloc try/except (gestion d'erreurs).
62	`        from firebase_admin import messaging`	Import de module ou symbole Python.
63	`        message = messaging.Message(`	Affectation de variable ou constante.
64	`            notification=messaging.Notification(title=title, body=body),`	Affectation de variable ou constante.
65	`            token=fcm_token,`	Affectation de variable ou constante.
66	`        )`	Instruction Python.
67	`        messaging.send(message)`	Instruction Python.
68	`        return True`	Retourne une valeur au appelant.
69	`    except Exception as e:`	Capture une exception.
70	`        print(f"[Firebase] push échoué : {e}")`	Instruction Python.
71	`        return False`	Retourne une valeur au appelant.

L.	Code	Explication
72		Ligne vide (séparation visuelle).

L.	Code	Explication
73		Ligne vide (séparation visuelle).
74	`def update_user_password(uid, new_password):`	Déclaration de la fonction update_user_password().
75	`    """Met à jour le mot de passe d'un utilisateur Firebase (côté admin).`	Commentaire / docstring : """Met à jour le mot de passe d'un utilisateur Firebase (côté admin).
76		Ligne vide (séparation visuelle).
77	`    Renvoie True si réussi. Lève une exception si Firebase n'est pas prêt ou en`	Instruction Python.
78	`    cas d'erreur (gérée par l'appelant).`	Instruction Python.
79	`    """`	Commentaire / docstring : """
80	`    if not _initialized:`	Condition : branche si la condition est vraie.
81	`        raise RuntimeError("Firebase non configuré")`	Instruction Python.
82	`    from firebase_admin import auth`	Import de module ou symbole Python.
83	`    auth.update_user(uid, password=new_password)`	Affectation de variable ou constante.
84	`    return True`	Retourne une valeur au appelant.

Fichier : agents/windows_agent.py

Agent Windows : presse-papier, navigateur, fichiers.

Classes déclarées

DownloadsHandler (ligne 189) — Classe Python.

Fonctions déclarées

url_a_analyser(url) — ligne 52
→ Retourne True si l'URL ouverte dans le navigateur mérite une analyse.
_timestamp_chrome_us() — ligne 63
→ Horodatage courant au format Chrome/Edge (microsecondes depuis 1601).
_timestamp_firefox_us() — ligne 68
→ Horodatage courant au format Firefox (microsecondes depuis 1970).
_chemins_navigateurs() — ligne 73
→ Chemins des bases d'historique des navigateurs installés.
_lire_visites_chromium(chemin_history, depuis_us) — ligne 98
→ Lit les visites récentes depuis History (Chrome, Edge, Brave).
_lire_visites_firefox(chemin_places, depuis_us) — ligne 124
→ Lit les visites récentes depuis places.sqlite (Firefox).
envoyer(event_type, content, metadata) — ligne 150
→ Envoie un rapport au serveur et affiche le verdict reçu.
merite_analyse(texte) — ligne 175
→ Pré-filtre simple pour éviter d'envoyer chaque copier-coller anodin.
surveiller_presse_papier() — ligne 199
→ Fonction utilitaire « surveiller_presse_papier ».
surveiller_navigateur() — ligne 217
→ Surveille les clics sur des liens via l'historique des navigateurs.
main() — ligne 262
→ Point d'entrée : initialise et démarre le programme.
on_created(self, event) — ligne 190
→ Fonction utilitaire « on_created ».

Explication ligne par ligne

L.	Code	Explication
1	`"""Agent Windows MoMoGuard — transparent et consenti.`	Commentaire / docstring : """Agent Windows MoMoGuard — transparent et consenti.
2		Ligne vide (séparation visuelle).
3	`Cet agent surveille TON propre environnement et envoie un rapport au serveur`	Instruction Python.
4	`quand il repère quelque chose qui mérite vérification :`	Instruction Python.
5		Ligne vide (séparation visuelle).
6	`  - le presse-papier (liens, codes USSD, numéros de compte copiés)`	Instruction Python.
7	`  - les liens ouverts dans le navigateur (Chrome, Edge, Firefox, Brave)`	Instruction Python.
8	`  - le dossier Téléchargements (nouveaux fichiers exécutables)`	Instruction Python.
9		Ligne vide (séparation visuelle).
10	`Il N'EST PAS furtif : il affiche dans la console tout ce qu'il fait et tu`	Instruction Python.
11	`peux l'arrêter à tout moment avec Ctrl+C.`	Instruction Python.
12		Ligne vide (séparation visuelle).
13	`Dépendances :  pip install requests pyperclip watchdog`	Instruction Python.
14	`"""`	Commentaire / docstring : """
15		Ligne vide (séparation visuelle).
16	`import os`	Import de module ou symbole Python.
17	`import shutil`	Import de module ou symbole Python.

L.	Code	Explication
18	`import sqlite3`	Import de module ou symbole Python.

L.	Code	Explication
19	`import tempfile`	Import de module ou symbole Python.
20	`import threading`	Import de module ou symbole Python.
21	`import time`	Import de module ou symbole Python.
22	`import requests`	Import de module ou symbole Python.
23	`import pyperclip`	Import de module ou symbole Python.
24	`from collections import deque`	Import de module ou symbole Python.
25	`from watchdog.observers import Observer`	Import de module ou symbole Python.
26	`from watchdog.events import FileSystemEventHandler`	Import de module ou symbole Python.
27		Ligne vide (séparation visuelle).
28	`# --- Configuration ---------------------

L.	Code	Explication
60	`    return lu.startswith("http://") or lu.startswith("https://")`	Retourne une valeur au appelant.
61		Ligne vide (séparation visuelle).
62		Ligne vide (séparation visuelle).
63	`def _timestamp_chrome_us():`	Déclaration de la fonction _timestamp_chrome_us().
64	`    """Horodatage courant au format Chrome/Edge (microsecondes depuis 1601)."""`	Commentaire / docstring : """Horodatage courant au format Chrome/Edge (microsecondes depuis 16…
65	`    return int((time.time() + _EPOQUE_WEBKIT) * 1_000_000)`	Retourne une valeur au appelant.
66		Ligne vide (séparation visuelle).
67		Ligne vide (séparation visuelle).
68	`def _timestamp_firefox_us():`	Déclaration de la fonction _timestamp_firefox_us().
69	`    """Horodatage courant au format Firefox (microsecondes depuis 1970)."""`	Commentaire / docstring : """Horodatage courant au format Firefox (microsecondes depuis 1970)."""…
70	`    return int(time.time() * 1_000_000)`	Retourne une valeur au appelant.
71		Ligne vide (séparation visuelle).
72		Ligne vide (séparation visuelle).

L.	Code	Explication
73	`def _chemins_navigateurs():`	Déclaration de la fonction _chemins_navigateurs().
74	`    """Chemins des bases d'historique des navigateurs installés."""`	Commentaire / docstring : """Chemins des bases d'historique des navigateurs installés."""
75	`    local = os.environ.get("LOCALAPPDATA", "")`	Affectation de variable ou constante.
76	`    appdata = os.environ.get("APPDATA", "")`	Affectation de variable ou constante.
77	`    chemins = []`	Affectation de variable ou constante.
78		Ligne vide (séparation visuelle).
79	`    for nom, rel in (`	Boucle de répétition.
80	`        ("chrome", os.path.join("Google", "Chrome", "User Data", "Default", "History")),`	Instruction Python.
81	`        ("edge", os.path.join("Microsoft", "Edge", "User Data", "Default", "History")),`	Instruction Python.
82	`        ("brave", os.path.join("BraveSoftware", "Brave-Browser", "User Data", "Default", "Histo…`	Instruction Python.
83	`    ):`	Instruction Python.
84	`        chemin = os.path.join(local, rel)`	Affectation de variable ou constante.
85	`        if os.path.isfile(chemin):`	Condition : branche si la condition est vraie.
86	`            chemins.append(("chromium", nom, chemin))`	Instruction Python.
87		Ligne vide (séparation visuelle).
88	`    profils = os.path.join(appdata, "Mozilla", "Firefox", "Profiles")`	Affectation de variable ou constante.
89	`    if os.path.isdir(profils):`	Condition : branche si la condition est vraie.
90	`        for profil in os.listdir(profils):`	Boucle de répétition.

L.	Code	Explication
91	`            places = os.path.join(profils, profil, "places.sqlite")`	Affectation de variable ou constante.
92	`            if os.path.isfile(places):`	Condition : branche si la condition est vraie.
93	`                chemins.append(("firefox", profil, places))`	Instruction Python.
94		Ligne vide (séparation visuelle).
95	`    return chemins`	Retourne une valeur au appelant.
96		Ligne vide (séparation visuelle).
97		Ligne vide (séparation visuelle).
98	`def _lire_visites_chromium(chemin_history, depuis_us):`	Déclaration de la fonction _lire_visites_chromium().
99	`    """Lit les visites récentes depuis History (Chrome, Edge, Brave)."""`	Commentaire / docstring : """Lit les visites récentes depuis History (Chrome, Edge, Brave)."""
100	`    tmp = tempfile.NamedTemporaryFile(suffix=".db", delete=False)`	Affectation de variable ou constante.
101	`    tmp.close()`	Instruction Python.
102	`    try:`	Début bloc try/except (gestion d'erreurs).

L.	Code	Explication
103	`        shutil.copy2(chemin_history, tmp.name)`	Instruction Python.
104	`        with sqlite3.connect(tmp.name) as conn:`	Context manager (ressource ouverte/fermée automatiquement).
105	`            return conn.execute(`	Retourne une valeur au appelant.
106	`                """`	Commentaire / docstring : """
107	`                SELECT u.url, COALESCE(u.title, ''), v.visit_time`	Instruction Python.
108	`                FROM visits v`	Instruction Python.

L.	Code	Explication
109	`                JOIN urls u ON u.id = v.url`	Affectation de variable ou constante.
110	`                WHERE v.visit_time > ?`	Instruction Python.
111	`                ORDER BY v.visit_time ASC`	Instruction Python.
112	`                """,`	Commentaire / docstring : """,
113	`                (depuis_us,),`	Instruction Python.
114	`            ).fetchall()`	Instruction Python.
115	`    except (sqlite3.Error, OSError):`	Capture une exception.
116	`        return []`	Retourne une valeur au appelant.
117	`    finally:`	Instruction Python.
118	`        try:`	Début bloc try/except (gestion d'erreurs).
119	`            os.unlink(tmp.name)`	Instruction Python.
120	`        except OSError:`	Capture une exception.
121	`            pass`	Instruction Python.
122		Ligne vide (séparation visuelle).
123		Ligne vide (séparation visuelle).
124	`def _lire_visites_firefox(chemin_places, depuis_us):`	Déclaration de la fonction _lire_visites_firefox().
125	`    """Lit les visites récentes depuis places.sqlite (Firefox)."""`	Commentaire / docstring : """Lit les visites récentes depuis places.sqlite (Firefox)."""
126	`    tmp = tempfile.NamedTemporaryFile(suffix=".db", delete=False)`	Affectation de variable ou constante.

L.	Code	Explication
127	`    tmp.close()`	Instruction Python.
128	`    try:`	Début bloc try/except (gestion d'erreurs).
129	`        shutil.copy2(chemin_places, tmp.name)`	Instruction Python.
130	`        with sqlite3.connect(tmp.name) as conn:`	Context manager (ressource ouverte/fermée automatiquement).
131	`            return conn.execute(`	Retourne une valeur au appelant.
132	`                """`	Commentaire / docstring : """
133	`                SELECT h.url, COALESCE(h.title, ''), v.visit_date`	Instruction Python.
134	`                FROM moz_historyvisits v`	Instruction Python.
135	`                JOIN moz_places h ON h.id = v.place_id`	Affectation de variable ou constante.
136	`                WHERE v.visit_date > ?`	Instruction Python.
137	`                ORDER BY v.visit_date ASC`	Instruction Python.
138	`                """,`	Commentaire / docstring : """,
139	`                (depuis_us,),`	Instruction Python.
140	`            ).fetchall()`	Instruction Python.
141	`    except (sqlite3.Error, OSError):`	Capture une exception.
142	`        return []`	Retourne une valeur au appelant.
143	`    finally:`	Instruction Python.
144	`        try:`	Début bloc try/except (gestion d'erreurs).

L.	Code	Explication
145	`            os.unlink(tmp.name)`	Instruction Python.
146	`        except OSError:`	Capture une exception.
147	`            pass`	Instruction Python.
148		Ligne vide (séparation visuelle).
149		Ligne vide (séparation visuelle).
150	`def envoyer(event_type, content, metadata=None):`	Déclaration de la fonction envoyer().
151	`    """Envoie un rapport au serveur et affiche le verdict reçu."""`	Commentaire / docstring : """Envoie un rapport au serveur et affiche le verdict reçu."""
152	`    payload = {`	Affectation de variable ou constante.
153	`        "agent_id": AGENT_ID,`	Instruction Python.
154	`        "platform": "windows",`	Instruction Python.
155	`        "event_type": event_type,`	Instruction Python.
156	`        "content": content,`	Instruction Python.
157	`        "metadata": metadata or {},`	Instruction Python.
158	`        "token": AGENT_TOKEN,`	Instruction Python.
159	`    }`	Instruction Python.
160	`    try:`	Début bloc try/except (gestion d'erreurs).
161	`        r = requests.post(SERVER_URL, json=payload, timeout=5)`	Affectation de variable ou constante.
162	`        if r.status_code == 403:`	Condition : branche si la condition est vraie.

L.	Code	Explication
163	`            print("  !! cet agent est BLOQUÉ par l'administrateur")`	Instruction Python.
164	`            return`	Instruction Python.
165	`        if r.status_code == 401:`	Condition : branche si la condition est vraie.
166	`            print("  !! jeton d'agent invalide (vérifie AGENT_TOKEN)")`	Instruction Python.
167	`            return`	Instruction Python.
168	`        data = r.json()`	Affectation de variable ou constante.
169	`        print(f"  -> envoyé [{event_type}] niveau={data.get('level')}")`	Affectation de variable ou constante.
170	`              f"score={data.get('score')}")`	Affectation de variable ou constante.
171	`    except Exception as e:`	Capture une exception.
172	`        print(f"  !! échec d'envoi : {e}")`	Instruction Python.
173		Ligne vide (séparation visuelle).
174		Ligne vide (séparation visuelle).
175	`def merite_analyse(texte):`	Déclaration de la fonction merite_analyse().
176	`    """Pré-filtre simple pour éviter d'envoyer chaque copier-coller anodin."""`	Commentaire / docstring : """Pré-filtre simple pour éviter d'envoyer chaque copier-coller anodin."""
177	`    t = texte.lower()`	Affectation de variable ou constante.
178	`    if "http://" in t or "https://" in t or "www." in t:`	Condition : branche si la condition est vraie.
179	`        return "link"`	Retourne une valeur au appelant.
180	`    if "*" in texte and "#" in texte:`	Condition : branche si la condition est vraie.

L.	Code	Explication
181	`        return "ussd"`	Retourne une valeur au appelant.
182	`    if any(c.isdigit() for c in texte) and sum(c.isdigit() for c in texte) > 8:`	Condition : branche si la condition est vraie.
183	`        return "clipboard"`	Retourne une valeur au appelant.
184	`    if any(k in t for k in MOTS_CLES):`	Condition : branche si la condition est vraie.
185	`        return "message"`	Retourne une valeur au appelant.
186	`    return None`	Retourne une valeur au appelant.
187		Ligne vide (séparation visuelle).

L.	Code	Explication
188		Ligne vide (séparation visuelle).
189	`class DownloadsHandler(FileSystemEventHandler):`	Déclaration de la classe DownloadsHandler.
190	`    def on_created(self, event):`	Déclaration de la fonction on_created().
191	`        if event.is_directory:`	Condition : branche si la condition est vraie.
192	`            return`	Instruction Python.
193	`        nom = os.path.basename(event.src_path)`	Affectation de variable ou constante.
194	`        if nom.lower().endswith(EXT_DANGEREUSES):`	Condition : branche si la condition est vraie.
195	`            print(f"[fichier] nouveau fichier exécutable : {nom}")`	Instruction Python.
196	`            envoyer("file", nom, {"path": event.src_path})`	Instruction Python.
197		Ligne vide (séparation visuelle).
198		Ligne vide (séparation visuelle).

L.	Code	Explication
199	`def surveiller_presse_papier():`	Déclaration de la fonction surveiller_presse_papier().
200	`    print(f"[presse-papier] surveillance active (toutes les {POLL_SECONDS}s)")`	Instruction Python.
201	`    dernier = ""`	Affectation de variable ou constante.
202	`    while True:`	Boucle de répétition.
203	`        try:`	Début bloc try/except (gestion d'erreurs).
204	`            actuel = pyperclip.paste()`	Affectation de variable ou constante.
205	`        except Exception:`	Capture une exception.
206	`            actuel = ""`	Affectation de variable ou constante.
207	`        if actuel and actuel != dernier:`	Condition : branche si la condition est vraie.
208	`            dernier = actuel`	Affectation de variable ou constante.
209	`            type_event = merite_analyse(actuel)`	Affectation de variable ou constante.
210	`            if type_event:`	Condition : branche si la condition est vraie.
211	`                apercu = actuel[:80].replace("\n", " ")`	Affectation de variable ou constante.
212	`                print(f"[presse-papier] détecté ({type_event}) : {apercu}")`	Instruction Python.
213	`                envoyer(type_event, actuel)`	Instruction Python.
214	`        time.sleep(POLL_SECONDS)`	Instruction Python.
215		Ligne vide (séparation visuelle).
216		Ligne vide (séparation visuelle).

L.	Code	Explication
217	`def surveiller_navigateur():`	Déclaration de la fonction surveiller_navigateur().
218	`    """Surveille les clics sur des liens via l'historique des navigateurs."""`	Commentaire / docstring : """Surveille les clics sur des liens via l'historique des navigateurs."""
219	`    print(f"[navigateur] surveillance des liens ouverts (toutes les {BROWSER_POLL_SECONDS}s)")`	Instruction Python.
220	`    chemins = _chemins_navigateurs()`	Affectation de variable ou constante.
221	`    if not chemins:`	Condition : branche si la condition est vraie.
222	`        print("[navigateur] aucun navigateur compatible détecté (Chrome, Edge, Firefox, Brave)")`	Instruction Python.
223	`        return`	Instruction Python.
224		Ligne vide (séparation visuelle).
225	`    for moteur, nom, _ in chemins:`	Boucle de répétition.
226	`        print(f"[navigateur] profil actif : {nom} ({moteur})")`	Instruction Python.
227		Ligne vide (séparation visuelle).
228	`    # Ne remonter que les visites après le démarrage de l'agent`	Commentaire / docstring : Ne remonter que les visites après le démarrage de l'agent
229	`    curseurs = {}`	Affectation de variable ou constante.
230	`    for moteur, nom, _ in chemins:`	Boucle de répétition.

L.	Code	Explication
231	`        cle = f"{moteur}:{nom}"`	Affectation de variable ou constante.
232	`        curseurs[cle] = (_timestamp_chrome_us() if moteur == "chromium"`	Affectation de variable ou constante.
233	`                         else _timestamp_firefox_us())`	Instruction Python.
234		Ligne vide (séparation visuelle).

L.	Code	Explication
235	`    urls_recentes = deque(maxlen=200)`	Affectation de variable ou constante.
236		Ligne vide (séparation visuelle).
237	`    while True:`	Boucle de répétition.
238	`        for moteur, nom, chemin in chemins:`	Boucle de répétition.
239	`            cle = f"{moteur}:{nom}"`	Affectation de variable ou constante.
240	`            depuis = curseurs[cle]`	Affectation de variable ou constante.
241	`            if moteur == "chromium":`	Condition : branche si la condition est vraie.
242	`                visites = _lire_visites_chromium(chemin, depuis)`	Affectation de variable ou constante.
243	`            else:`	Branche alternative (sinon).
244	`                visites = _lire_visites_firefox(chemin, depuis)`	Affectation de variable ou constante.
245		Ligne vide (séparation visuelle).
246	`            for url, titre, horodatage in visites:`	Boucle de répétition.
247	`                curseurs[cle] = max(curseurs[cle], horodatage)`	Affectation de variable ou constante.
248	`                if not url_a_analyser(url) or url in urls_recentes:`	Condition : branche si la condition est vraie.
249	`                    continue`	Instruction Python.
250	`                urls_recentes.append(url)`	Instruction Python.
251	`                apercu = url[:80]`	Affectation de variable ou constante.
252	`                print(f"[navigateur] lien ouvert ({nom}) : {apercu}")`	Instruction Python.

L.	Code	Explication
253	`                envoyer("link", url, {`	Instruction Python.
254	`                    "source": "browser_click",`	Instruction Python.
255	`                    "browser": nom,`	Instruction Python.
256	`                    "title": titre[:200] if titre else "",`	Instruction Python.
257	`                })`	Instruction Python.
258		Ligne vide (séparation visuelle).
259	`        time.sleep(BROWSER_POLL_SECONDS)`	Instruction Python.
260		Ligne vide (séparation visuelle).
261		Ligne vide (séparation visuelle).
262	`def main():`	Déclaration de la fonction main().
263	`    print("=" * 60)`	Affectation de variable ou constante.
264	`    print(" Agent Windows MoMoGuard (transparent)")`	Instruction Python.
265	`    print(f" Serveur : {SERVER_URL}")`	Instruction Python.
266	`    print(f" Dossier surveillé : {DOWNLOADS_DIR}")`	Instruction Python.
267	`    print(" Arrêt : Ctrl+C")`	Instruction Python.
268	`    print("=" * 60)`	Affectation de variable ou constante.
269		Ligne vide (séparation visuelle).
270	`    observer = None`	Affectation de variable ou constante.

L.	Code	Explication
271	`    if os.path.isdir(DOWNLOADS_DIR):`	Condition : branche si la condition est vraie.
272	`        observer = Observer()`	Affectation de variable ou constante.

L.	Code	Explication
273	`        observer.schedule(DownloadsHandler(), DOWNLOADS_DIR, recursive=False)`	Affectation de variable ou constante.
274	`        observer.start()`	Instruction Python.
275	`        print(f"[fichier] surveillance de {DOWNLOADS_DIR}")`	Instruction Python.
276	`    else:`	Branche alternative (sinon).
277	`        print("[fichier] dossier Téléchargements introuvable — surveillance fichiers désactivée...")`	Instruction Python.
278		Ligne vide (séparation visuelle).
279	`    threading.Thread(target=surveiller_navigateur, daemon=True).start()`	Affectation de variable ou constante.
280		Ligne vide (séparation visuelle).
281	`    try:`	Début bloc try/except (gestion d'erreurs).
282	`        surveiller_presse_papier()`	Instruction Python.
283	`    except KeyboardInterrupt:`	Capture une exception.
284	`        print("\n[arrêt] agent stoppé par l'utilisateur")`	Instruction Python.
285	`    finally:`	Instruction Python.
286	`        if observer:`	Condition : branche si la condition est vraie.
287	`            observer.stop()`	Instruction Python.
288	`            observer.join()`	Instruction Python.

L.	Code	Explication
289		Ligne vide (séparation visuelle).
290		Ligne vide (séparation visuelle).
291	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
292	`    main()`	Instruction Python.

Fichier : server/static/dashboard.js

Dashboard temps réel (JavaScript).

Fonctions déclarées

escapeHtml(s) — ligne 4

→ Échappe le HTML pour éviter les injections XSS.

formatTime(iso) — ligne 10

→ Fonction utilitaire « formatTime ».

renderStats() — ligne 14

→ Met à jour les compteurs et barres du dashboard.

buildReport(r) — ligne 28

→ Construit l'élément HTML d'une alerte.

matchesFilters(node) — ligne 53

→ Fonction utilitaire « matchesFilters ».

applyFilters() — ligne 65

→ Fonction utilitaire « applyFilters ».

addReport(r, prepend) — ligne 71

→ Fonction utilitaire « addReport ».

bumpStats(level) — ligne 81

→ Fonction utilitaire « bumpStats ».

Explication ligne par ligne

L.	Code	Explication
1	`const LEVEL_LABEL = { faible: "Faible", moyen: "Moyen", eleve: "Élevé" };`	Déclaration de constante/variable.
2	`const stats = { total: 0, faible: 0, moyen: 0, eleve: 0 };`	Déclaration de constante/variable.
3		Ligne vide.
4	`function escapeHtml(s) {`	Déclaration fonction JavaScript escapeHtml.
5	`  return String(s)`	Instruction JavaScript.
6	`    .replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">")`	Instruction JavaScript.
7	`    .replace(/"/g, """).replace(/'/g, "'");`	Instruction JavaScript.
8	`}`	Instruction JavaScript.
9		Ligne vide.
10	`function formatTime(iso) {`	Déclaration fonction JavaScript formatTime.
11	`  return new Date(iso).toLocaleString("fr-FR", { hour12: false });`	Instruction JavaScript.
12	`}`	Instruction JavaScript.
13		Ligne vide.
14	`function renderStats() {`	Déclaration fonction JavaScript renderStats.
15	`  document.getElementById("stat-total").textContent = stats.total;`	Manipulation du DOM (page HTML).
16	`  document.getElementById("stat-faible").textContent = stats.faible;`	Manipulation du DOM (page HTML).
17	`  document.getElementById("stat-moyen").textContent = stats.moyen;`	Manipulation du DOM (page HTML).
18	`  document.getElementById("stat-eleve").textContent = stats.eleve;`	Manipulation du DOM (page HTML).

L.	Code	Explication
19	`  const t = stats.total || 1;`	Déclaration de constante/variable.
20	`  [["faible", stats.faible], ["moyen", stats.moyen], ["eleve", stats.eleve]].forEach(([lvl, n])…`	Instruction JavaScript.
21	`    const bar = document.getElementById("bar-" + lvl);`	Déclaration de constante/variable.
22	`    const cnt = document.getElementById("barc-" + lvl);`	Déclaration de constante/variable.
23	`    if (bar) bar.style.width = Math.round((n / t) * 100) + "%";`	Instruction JavaScript.
24	`    if (cnt) cnt.textContent = n;`	Instruction JavaScript.
25	`  });`	Instruction JavaScript.

L.	Code	Explication
26	`}`	Instruction JavaScript.
27		Ligne vide.
28	`function buildReport(r) {`	Déclaration fonction JavaScript buildReport.
29	`  const li = document.createElement("li");`	Déclaration de constante/variable.
30	`  li.className = "report " + r.level;`	Instruction JavaScript.
31	`  li.dataset.level = r.level;`	Instruction JavaScript.
32	`  li.dataset.platform = (r.platform || "").toLowerCase();`	Instruction JavaScript.
33	`  li.dataset.agent = (r.agent_id || "").toLowerCase();`	Instruction JavaScript.
34	`  li.dataset.text = ((r.content || "") + " " + (r.reasons || []).join(" ")).toLowerCase();`	Instruction JavaScript.
35	`  const reasons = (r.reasons || []).map((x) => ` ${escapeHtml(x)} `).join("");`	Déclaration de fonction JavaScript (x) ; Déclaration de constante.
36	` li.innerHTML = ``	Injection HTML dans un élément (alerte affichée).

L.	Code	Explication
37	`    <div class="report-top">`	Instruction JavaScript.
38	`      <div class="report-meta">`	Instruction JavaScript.
39	`        <span class="tag">${escapeHtml(r.platform)}</span>`	Instruction JavaScript.
40	`        <span class="tag">${escapeHtml(r.event_type)}</span>`	Instruction JavaScript.
41	`        <span>${escapeHtml(r.agent_id)}</span>`	Instruction JavaScript.
42	`        <span>${formatTime(r.created_at)}</span>`	Instruction JavaScript.
43	`      </div>`	Instruction JavaScript.
44	`      <span class="badge ${r.level}">${LEVEL_LABEL[r.level]} · ${r.score}</span>`	Instruction JavaScript.
45	`    </div>`	Instruction JavaScript.
46	`    <div class="report-content">${escapeHtml(r.content)}</div>`	Instruction JavaScript.
47	`    <ul class="reasons">${reasons}</ul>`	Instruction JavaScript.
48	`    ${r.conseil ? ` Conseil : ${escapeHtml(r.consei…`	Instruction JavaScript.
49	`  `;`	Instruction JavaScript.
50	`  return li;`	Instruction JavaScript.
51	`}`	Instruction JavaScript.
52		Ligne vide.
53	`function matchesFilters(node) {`	Déclaration fonction JavaScript matchesFilters.
54	`  const level = document.getElementById("filter-level").value;`	Déclaration de constante/variable.

L.	Code	Explication
55	`  const platform = document.getElementById("filter-platform").value.toLowerCase();`	Déclaration de constante/variable.
56	`  const agent = document.getElementById("filter-agent").value.toLowerCase();`	Déclaration de constante/variable.
57	`  const search = document.getElementById("filter-search").value.trim().toLowerCase();`	Déclaration de constante/variable.
58	`  if (level && node.dataset.level !== level) return false;`	Instruction JavaScript.
59	`  if (platform && node.dataset.platform !== platform) return false;`	Instruction JavaScript.
60	`  if (agent && !node.dataset.agent.includes(agent)) return false;`	Instruction JavaScript.
61	`  if (search && !node.dataset.text.includes(search)) return false;`	Instruction JavaScript.
62	`  return true;`	Instruction JavaScript.
63	`}`	Instruction JavaScript.
64		Ligne vide.
65	`function applyFilters() {`	Déclaration fonction JavaScript applyFilters.
66	`  document.querySelectorAll("#feed .report").forEach((node) => {`	Manipulation du DOM (page HTML).
67	`    node.style.display = matchesFilters(node) ? "" : "none";`	Instruction JavaScript.
68	`  });`	Instruction JavaScript.

L.	Code	Explication
69	`}`	Instruction JavaScript.
70		Ligne vide.
71	`function addReport(r, prepend) {`	Déclaration fonction JavaScript addReport.
72	`  const feed = document.getElementById("feed");`	Déclaration de constante/variable.

L.	Code	Explication
73	`  const empty = document.getElementById("empty-state");`	Déclaration de constante/variable.
74	`  if (empty) empty.remove();`	Instruction JavaScript.
75	`  const node = buildReport(r);`	Déclaration de constante/variable.
76	`  if (prepend) feed.prepend(node);`	Instruction JavaScript.
77	`  else feed.appendChild(node);`	Instruction JavaScript.
78	`  node.style.display = matchesFilters(node) ? "" : "none";`	Instruction JavaScript.
79	`}`	Instruction JavaScript.
80		Ligne vide.
81	`function bumpStats(level) {`	Déclaration fonction JavaScript bumpStats.
82	`  stats.total += 1;`	Instruction JavaScript.
83	`  if (stats[level] !== undefined) stats[level] += 1;`	Instruction JavaScript.
84	`  renderStats();`	Instruction JavaScript.
85	`}`	Instruction JavaScript.
86		Ligne vide.
87	`["filter-level", "filter-platform", "filter-agent", "filter-search"].forEach((id) => {`	Instruction JavaScript.
88	`  const el = document.getElementById(id);`	Déclaration de constante/variable.
89	`  if (el) el.addEventListener("input", applyFilters);`	Écouteur d'événement utilisateur (clic, saisie…).
90	`});`	Instruction JavaScript.

L.	Code	Explication
91		Ligne vide.
92	`// --- Chargement initial ----------`	Commentaire : --- Chargement initial ----------
93	`fetch("/api/reports")`	Requête HTTP asynchrone vers le serveur.
94	`  .then((res) => res.json())`	Suite d'une promesse asynchrone.
95	`  .then((data) => {`	Suite d'une promesse asynchrone.
96	`    if (data.stats) { Object.assign(stats, data.stats); renderStats(); }`	Instruction JavaScript.
97	`    (data.reports || []).forEach((r) => addReport(r, false));`	Instruction JavaScript.
98	`  })`	Instruction JavaScript.
99	`  .catch((e) => console.error("Erreur de chargement initial", e));`	Suite d'une promesse asynchrone.
100		Ligne vide.
101	`// --- Temps réel ----------`	Commentaire : --- Temps réel ----------
102	`const socket = io();`	Déclaration de constante/variable.
103	`const statusEl = document.getElementById("conn-status");`	Déclaration de constante/variable.
104	`const labelEl = document.getElementById("conn-label");`	Déclaration de constante/variable.
105		Ligne vide.
106	`socket.on("connect", () => {`	Gestion événement WebSocket temps réel.
107	`  statusEl.classList.add("online"); statusEl.classList.remove("offline");`	Instruction JavaScript.
108	`  labelEl.textContent = "en ligne";`	Instruction JavaScript.

L.	Code	Explication
109	`});`	Instruction JavaScript.
110	`socket.on("disconnect", () => {`	Gestion événement WebSocket temps réel.

L.	Code	Explication
111	`  statusEl.classList.add("offline"); statusEl.classList.remove("online");`	Instruction JavaScript.
112	`  labelEl.textContent = "hors ligne";`	Instruction JavaScript.
113	`});`	Instruction JavaScript.
114	`socket.on("new_report", (r) => { addReport(r, true); bumpStats(r.severity); });`	Gestion d'événement WebSocket temps réel.
115	``	Ligne vide.
116	`// --- Bouton de démonstration (soutenance) ---`	Commentaire : --- Bouton de démonstration (soutenance) ---
117	`const demoBtn = document.getElementById("btn-demo");`	Déclaration de constante/variable.
118	`if (demoBtn) {`	Instruction JavaScript.
119	`  demoBtn.addEventListener("click", () => {`	Écouteur d'événement utilisateur (clic, saisie…).
120	`    const token = (document.querySelector('meta[name="csrf-token"]') || {}).content || "";`	Déclaration de constante/variable.
121	`    demoBtn.disabled = true;`	Instruction JavaScript.
122	`    fetch("/demo", { method: "POST", headers: { "X-CSRFToken": token } })`	Requête HTTP asynchrone vers le serveur.
123	`      .then((r) => r.json())`	Suite d'une promesse asynchrone.
124	`      .catch((e) => console.error("Démo :", e))`	Suite d'une promesse asynchrone.
125	`      .finally(() => { demoBtn.disabled = false; });`	Instruction JavaScript.
126	`  });`	Instruction JavaScript.

L.	Code	Explication
127	`}`	Instruction JavaScript.

Fichier : scripts/seed_demo.py

Insertion de données de démonstration.

Fonctions déclarées

main() — ligne 16

→ Point d'entrée : initialise et démarre le programme.

Explication ligne par ligne

L.	Code	Explication
1	`"""Insère un jeu de données de démonstration (utilisateurs, agents, alertes)`	Commentaire / docstring : """Insère un jeu de données de démonstration (utilisateurs, agents, alertes)
2		Ligne vide (séparation visuelle).
3	`Usage : python scripts/seed_demo.py`	Instruction Python.
4	`"""`	Commentaire / docstring : """
5	`import os, sys`	Import de module ou symbole Python.
6	`sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))`	Instruction Python.
7	`from server import database, scoring`	Import de module ou symbole Python.
8		Ligne vide (séparation visuelle).
9	`SAMPLES = [`	Affectation de variable ou constante.
10	`("sms", "URGENT votre compte Orange Money est bloque, reactivez sur http://0range-money.tk"…`	Instruction Python.
11	`("sms", "Felicitations ! Vous avez gagne 5000$, envoyez vos infos pour recevoir", "Promo"),`	Instruction Python.
12	`("sms", "Votre carte SIM va etre remplacee, communiquez votre code PUK", "Operateur"),`	Instruction Python.
13	`("sms", "Bonjour, on se voit demain a 15h", "Maman"),`	Instruction Python.
14	`]`	Instruction Python.
15		Ligne vide (séparation visuelle).
16	`def main():`	Déclaration de la fonction main().
17	`database.init_db()`	Appel à la couche base de données.
18	`database.upsert_user("demo_pending", "client.attente@demo.cd", "Client En Attente")`	Appel à la couche base de données.

L.	Code	Explication
19	`database.upsert_user("demo_ok", "client.ok@demo.cd", "Client Approuve", ip="41.243.10.7")`	Appel à la couche base de données.
20	`database.set_user_status("demo_ok", "approved")`	Appel à la couche base de données.
21		Ligne vide (séparation visuelle).
22	`database.register_agent("android-demo", "Telephone de demo", "android")`	Appel à la couche base de données.
23	`database.register_agent("windows-demo", "PC de demo", "windows")`	Appel à la couche base de données.
24	`database.touch_agent("android-demo")`	Appel à la couche base de données.
25		Ligne vide (séparation visuelle).
26	`thr = database.get_thresholds()`	Appel à la couche base de données.
27	`for etype, content, sender in SAMPLES:`	Boucle de répétition.
28	`score, level, reasons = scoring.score_event(etype, content, {"sender": sender}, thr)`	Appel à la couche base de données.
29	`database.insert_report("android-demo", "android", etype, content,`	Appel à la couche base de données.
30	`{"sender": sender}, score, level, reasons)`	Affectation de variable ou constante.
31	`print(f"Données de démonstration insérées ({len(SAMPLES)} alertes).")`	Instruction Python.
32		Ligne vide (séparation visuelle).
33	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
34	`main()`	Instruction Python.

Fichier : scripts/backup_db.py

Sauvegarde JSON de la base.

Fonctions déclarées

main() — ligne 10

→ Point d'entrée : initialise et démarre le programme.

Explication ligne par ligne

L.	Code	Explication
1	`"""Sauvegarde portable de la base (toutes tables -> JSON horodaté)."""`	Commentaire / docstring : """Sauvegarde portable de la base (toutes tables -> JSON horodaté).
2		Ligne vide (séparation visuelle).
3	`Fonctionne quel que soit le moteur (SQLite, MySQL, PostgreSQL).`	Instruction Python.
4	`Usage : python scripts/backup_db.py`	Instruction Python.
5	`"""`	Commentaire / docstring : """
6	`import os, sys, json, datetime`	Import de module ou symbole Python.
7	`sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))`	Instruction Python.
8	`from server import database`	Import de module ou symbole Python.
9		Ligne vide (séparation visuelle).
10	`def main():`	Déclaration de la fonction main().
11	`    database.init_db()`	Appel à la couche base de données.
12	`    os.makedirs("backups", exist_ok=True)`	Affectation de variable ou constante.
13	`    dump = {}`	Affectation de variable ou constante.
14	`    with database.engine.connect() as conn:`	Context manager (ressource ouverte/fermée automatiquement).
15	`        for table in database.md.sorted_tables:`	Boucle de répétition.
16	`            rows = conn.execute(database.select(table)).mappings().all()`	Appel à la couche base de données.
17	`            dump[table.name] = [dict(r) for r in rows]`	Affectation de variable ou constante.
18	`    ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")`	Affectation de variable ou constante.

L.	Code	Explication
19	`    path = os.path.join("backups", f"momoguard_backup_{ts}.json")`	Affectation de variable ou constante.
20	`    with open(path, "w", encoding="utf-8") as f:`	Context manager (ressource ouverte/fermée automatiquement).
21	`        json.dump(dump, f, ensure_ascii=False, indent=2, default=str)`	Affectation de variable ou constante.
22	`    total = sum(len(v) for v in dump.values())`	Affectation de variable ou constante.
23	`    print(f"Sauvegarde écrite : {path} ({total} lignes)")`	Instruction Python.
24		Ligne vide (séparation visuelle).
25	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
26	`    main()`	Instruction Python.

Fichier : evaluation/evaluer.py

Évaluation chiffrée du moteur de scoring.

Fonctions déclarées

predire(text, etype) — ligne 20

→ Fonction utilitaire « predire ».

main() — ligne 25

→ Point d'entrée : initialise et démarre le programme.

_ecrire_rapport(total, tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude, erreurs) — ligne 76

→ Fonction utilitaire « _ecrire_rapport ».

_figures(tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude) — ligne 106

→ Fonction utilitaire « _figures ».

Explication ligne par ligne

L.	Code	Explication
1	`"""Évaluation chiffrée du moteur de détection MoMoGuard.`	Commentaire / docstring : """Évaluation chiffrée du moteur de détection MoMoGuard.
2		Ligne vide (séparation visuelle).
3	`Fait passer un jeu de messages étiquetés dans le moteur de scoring et calcule`	Instruction Python.
4	`le taux de détection, le taux de faux positifs, la précision, le rappel, le`	Instruction Python.
5	`F1-score et l'exactitude, avec matrice de confusion et figures.`	Instruction Python.
6		Ligne vide (séparation visuelle).
7	`Usage : python evaluation/evaluer.py`	Instruction Python.
8	`"""`	Commentaire / docstring : """
9	`import os`	Import de module ou symbole Python.
10	`import sys`	Import de module ou symbole Python.
11	`import json`	Import de module ou symbole Python.
12		Ligne vide (séparation visuelle).
13	`sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))`	Instruction Python.
14	`from server import scoring`	Import de module ou symbole Python.
15		Ligne vide (séparation visuelle).
16	`ICI = os.path.dirname(os.path.abspath(__file__))`	Affectation de variable ou constante.
17	`SEUILS = (25, 55)  # seuils par défaut (moyen, eleve)`	Affectation de variable ou constante.
18		Ligne vide (séparation visuelle).

L.	Code	Explication
19		Ligne vide (séparation visuelle).
20	`def predire(text, etype):`	Déclaration de la fonction predire().
21	`    _, level, _ = scoring.score_event(etype, text, {}, SEUILS)`	Appel au moteur de scoring.
22	`    return "fraude" if level in ("moyen", "eleve") else "legitime"`	Renvoie une valeur au appelant.
23		Ligne vide (séparation visuelle).
24		Ligne vide (séparation visuelle).
25	`def main():`	Déclaration de la fonction main().
26	`    with open(os.path.join(ICI, "dataset_evaluation.json"), encoding="utf-8") as f:`	Contexte manager (fermeture automatique).
27	`        data = json.load(f)`	Affectation de variable ou constante.
28		Ligne vide (séparation visuelle).
29	`    tp = fp = tn = fn = 0`	Affectation de variable ou constante.
30	`    erreurs = []`	Affectation de variable ou constante.
31	`    for item in data:`	Boucle de répétition.

L.	Code	Explication
32	`        pred = predire(item["text"], item.get("type", "smart"))`	Affectation de variable ou constante.
33	`        vrai = item["label"]`	Affectation de variable ou constante.
34	`        if vrai == "fraude" and pred == "fraude":`	Condition : branche si la condition est vraie.
35	`            tp += 1`	Affectation de variable ou constante.
36	`        elif vrai == "fraude" and pred == "legitime":`	Condition : branche si la condition est vraie.

L.	Code	Explication
37	`            fn += 1`	Affectation de variable ou constante.
38	`            erreurs.append(("Faux negatif (fraude manquee)", item["text"]))`	Instruction Python.
39	`        elif vrai == "legitime" and pred == "fraude":`	Condition : branche si la condition est vraie.
40	`            fp += 1`	Affectation de variable ou constante.
41	`            erreurs.append(("Faux positif (fausse alerte)", item["text"]))`	Instruction Python.
42	`        else:`	Branche alternative (sinon).
43	`            tn += 1`	Affectation de variable ou constante.
44		Ligne vide (séparation visuelle).
45	`    total = tp + fp + tn + fn`	Affectation de variable ou constante.
46	`    rappel = tp / (tp + fn) if (tp + fn) else 0      # taux de detection`	Affectation de variable ou constante.
47	`    tfp = fp / (fp + tn) if (fp + tn) else 0         # taux de faux positifs`	Affectation de variable ou constante.
48	`    precision = tp / (tp + fp) if (tp + fp) else 0`	Affectation de variable ou constante.
49	`    f1 = 2 * precision * rappel / (precision + rappel) if (precision + rappel) else 0`	Affectation de variable ou constante.
50	`    exactitude = (tp + tn) / total if total else 0`	Affectation de variable ou constante.
51		Ligne vide (séparation visuelle).
52	`    print("=" * 56)`	Affectation de variable ou constante.
53	`    print(" Évaluation du moteur de détection MoMoGuard")`	Instruction Python.
54	`    print("=" * 56)`	Affectation de variable ou constante.

L.	Code	Explication
55	`    print(f" Échantillon         : {total} messages ({tp+fn} fraude, {tn+fp} légitime)")`	Instruction Python.
56	`    print(f" Seuils (moyen/élevé): {SEUILS[0]} / {SEUILS[1]}")`	Instruction Python.
57	`    print("-" * 56)`	Instruction Python.
58	`    print(f" Vrais positifs  (TP): {tp:3d}    Faux negatifs (FN): {fn:3d}")`	Instruction Python.
59	`    print(f" Vrais negatifs  (TN): {tn:3d}    Faux positifs (FP): {fp:3d}")`	Instruction Python.
60	`    print("-" * 56)`	Instruction Python.
61	`    print(f" Taux de detection (rappel)  : {rappel:6.1%}")`	Instruction Python.
62	`    print(f" Taux de faux positifs       : {tfp:6.1%}")`	Instruction Python.
63	`    print(f" Precision                   : {precision:6.1%}")`	Instruction Python.
64	`    print(f" F1-score                    : {f1:6.1%}")`	Instruction Python.
65	`    print(f" Exactitude (accuracy)       : {exactitude:6.1%}")`	Instruction Python.
66	`    print("=" * 56)`	Affectation de variable ou constante.
67	`    if erreurs:`	Condition : branche si la condition est vraie.
68	`        print(" Erreurs (analyse critique) :")`	Instruction Python.
69	`        for kind, txt in erreurs:`	Boucle de répétition.
70	`            print(f"  - {kind}: {txt[:60]}...")`	Instruction Python.
71		Ligne vide (séparation visuelle).
72	`    _ecrire_rapport(total, tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude, erreurs)`	Instruction Python.

L.	Code	Explication
73	`    _figures(tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude)`	Instruction Python.

L.	Code	Explication
74		Ligne vide (séparation visuelle).
75		Ligne vide (séparation visuelle).
76	`def _ecrire_rapport(total, tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude, seuils):`	Déclaration de la fonction _ecrire_rapport().
77	`    lignes = [`	Affectation de variable ou constante.
78	`        "# Évaluation chiffrée du moteur de détection MoMoGuard\n",`	Instruction Python.
79	`        "## Méthodologie\n",`	Instruction Python.
80	`        f"Un jeu de **{total} messages** étiquetés ({tp+fn} frauduleux, {tn+fp} légitimes), "`	Instruction Python.
81	`        "représentatifs du contexte Mobile Money en RDC, a été soumis au moteur de scoring. "`	Instruction Python.
82	`        f"Un message est considéré comme détecté lorsque son niveau atteint « moyen » ou « élev…`	Instruction Python.
83	`        f"(seuils {SEUILS[0]}/{SEUILS[1]}).\n",`	Instruction Python.
84	`        "## Matrice de confusion\n",`	Instruction Python.
85	` "|	Prédit fraude	Prédit légitime |",`	Instruction Python.
86	`        "|---|---|---|",`	Instruction Python.
87	` f"| **Réel fraude**	{tp} (VP)	{fn} (FN) |",`	Instruction Python.
88	` f"| **Réel légitime**	{fp} (FP)	{tn} (VN) |\n",`	Instruction Python.
89	`        "## Résultats\n",`	Instruction Python.
90	` "| Métrique	Valeur |",`	Instruction Python.

L.	Code	Explication
91	`        "|---|---|",`	Instruction Python.
92	` f"| Taux de détection (rappel)	{rappel:.1%} |",`	Instruction Python.
93	` f"| Taux de faux positifs	{tfp:.1%} |",`	Instruction Python.
94	` f"| Précision	{precision:.1%} |",`	Instruction Python.
95	` f"| F1-score	{f1:.1%} |",`	Instruction Python.
96	` f"| Exactitude	{exactitude:.1%} |\n",`	Instruction Python.
97	`    ]`	Instruction Python.
98	`    if erreurs:`	Condition : branche si la condition est vraie.
99	`        lignes.append("## Analyse critique des erreurs\n")`	Instruction Python.
100	`        for kind, txt in erreurs:`	Boucle de répétition.
101	`            lignes.append(f"- **{kind}** : « {txt} »")`	Instruction Python.
102	`    with open(os.path.join(ICI, "resultats.md"), "w", encoding="utf-8") as f:`	Contexte manager (assure la fermeture de la ressource ouverte/fermée automatiquement).
103	`        f.write("\n".join(lignes) + "\n")`	Instruction Python.
104		Ligne vide (séparation visuelle).
105		Ligne vide (séparation visuelle).
106	`def _figures(tp, fp, tn, fn, rappel, tfp, precision, f1, exactitude):`	Déclaration de la fonction _figures().
107	`    import matplotlib`	Import de module ou symbole Python.
108	`    matplotlib.use("Agg")`	Instruction Python.

L.	Code	Explication
109	`    import matplotlib.pyplot as plt`	Import de module ou symbole Python.
110		Ligne vide (séparation visuelle).
111	`    # Matrice de confusion`	Commentaire / docstring : Matrice de confusion
112	`    fig, ax = plt.subplots(figsize=(5, 4))`	Affectation de variable ou constante.
113	`    mat = [[tp, fn], [fp, tn]]`	Affectation de variable ou constante.
114	`    ax.imshow(mat, cmap="Purples")`	Affectation de variable ou constante.
115	`    ax.set_xticks([0, 1]); ax.set_xticklabels(["Prédit\nfraude", "Prédit\nlégitime"])`	Instruction Python.
116	`    ax.set_yticks([0, 1]); ax.set_yticklabels(["Réel\nfraude", "Réel\nlégitime"])`	Instruction Python.

L.	Code	Explication
117	`    for i in range(2):`	Boucle de répétition.
118	`        for j in range(2):`	Boucle de répétition.
119	`            ax.text(j, i, mat[i][j], ha="center", va="center", fontsize=18,`	Affectation de variable ou constante.
120	`                    color="black")`	Affectation de variable ou constante.
121	`    ax.set_title("Matrice de confusion — MoMoGuard")`	Instruction Python.
122	`    fig.tight_layout()`	Instruction Python.
123	`    fig.savefig(os.path.join(ICI, "confusion_matrix.png"), dpi=130)`	Affectation de variable ou constante.
124		Ligne vide (séparation visuelle).
125	`    # Métriques`	Commentaire / docstring : Métriques
126	`    fig2, ax2 = plt.subplots(figsize=(6, 4))`	Affectation de variable ou constante.

L.	Code	Explication
127	`    noms = ["Détection", "Précision", "F1-score", "Exactitude"]`	Affectation de variable ou constante.
128	`    vals = [rappel, precision, f1, exactitude]`	Affectation de variable ou constante.
129	`    ax2.bar(noms, [v * 100 for v in vals], color="#6c63ff")`	Affectation de variable ou constante.
130	`    ax2.set_ylim(0, 100); ax2.set_ylabel("%")`	Instruction Python.
131	`    ax2.set_title("Performances du moteur de détection")`	Instruction Python.
132	`    for i, v in enumerate(vals):`	Boucle de répétition.
133	`        ax2.text(i, v * 100 + 1, f"{v:.0%}", ha="center", fontsize=11)`	Affectation de variable ou constante.
134	`    fig2.tight_layout()`	Instruction Python.
135	`    fig2.savefig(os.path.join(ICI, "metriques.png"), dpi=130)`	Affectation de variable ou constante.
136	`    print(" Figures écrites : evaluation/confusion_matrix.png, evaluation/metriques.png")`	Instruction Python.
137		Ligne vide (séparation visuelle).
138		Ligne vide (séparation visuelle).
139	`if __name__ == "__main__":`	Condition : branche si la condition est vraie.
140	`    main()`	Instruction Python.

Annexe — Autres fichiers du projet

server/templates/

Pages HTML Jinja2 : base (layout), dashboard, login, login_2fa, account, agents, users, user_detail, settings, consent, consents, audit, twofa_setup.

server/static/style.css

Feuille de styles CSS : thème sombre/clair, cartes stats, alertes, graphiques, formulaires.

tests/

Suite pytest : scoring, report, users, 2FA, audit, security, settings, threat_intel, demo_conseil, windows_agent.

tests/conftest.py

Fixtures pytest : instance Flask isolée sur SQLite temporaire.

evaluation/dataset_evaluation.json

48 messages étiquetés pour benchmark du moteur.

requirements.txt

Dépendances Python (Flask, SQLAlchemy, Firebase, pytest…).

Makefile

Raccourcis : install, run, test, seed, backup.